Manual de Programación Ladder-Arduino

Curso de Programación Ladder y Aplicación uPLC para Arduino

ALEJANDRO MARTIN TORRES

@ prof.martintorres@educ.ar

ETI Patagonia

Curso autodidacta de Automatización y Control Empleando uPLC (conversor LADDER-ARDUINO)

ETI Patagonia
Comodoro Rivadavia - Chubut
Patagonia Argentina
prof.martintorres@educ.ar

El campo de la automatización y control por instrumental, es un campo muy amplio y si bien, hasta hoy en día siguen evolucionando con las nuevas innovaciones tecnológicas, los principios del control, son siempre los mismos. Los avances en la tecnología software, nos han demostrado más de una vez, que los intérpretes de lenguaje son una estrategia para poder tener las herramientas a la mano a un bajo coste y un resultado bastante satisfactorio.
En el siguiente curso, pretendo introducirlos en los principios básicos del control, mediante las lógicas de contactos y bloques de funciones que tiene nuestra aplicación uPLC.

1 Introducción

Como ya mencioné en reiteradas ocasiones, el campo de la automatización y control, es un campo muy amplio al igual que el campo de la instrumentación. Este sistema de control de procesos, no tuvo su origen cuando aparecieron en el mercado los PLC. En si, no voy a emplazarme hasta aquellas épocas para contarles con detalles cual fue su evolución, pero tratare de esforzarme y ser breve para crearles una noción básica.
Esta es una ciencia que hace su aparición mucho antes de que el hombre descubriera la electricidad; si bien era bastante precaria, el ingenio del hombre, lo llevo a conocer y estudiar otras ciencias para aprender a emplearlas en sus automatismos (leyes de la física tales como palancas, poleas, niveles y hasta
presión y caudal).
La automatización, hasta hoy en día fue evolucionando con las nuevas innovaciones tecnológicas hasta tener lo que llamamos controladores lógicos programables de la era análoga-digital; pero hasta hace tan solo unas décadas atrás, el hombre implementa su ingenio nuevamente, y descubre que controlando el paso de la energía, se puede hacer automatismos con lógicas cableadas y contactos.
Los 1eros PLC, eran sencillamente eso, inmensos tableros llenos de relay y cables que estaban armados de tal manera que daban una secuencia de permisos y condiciones para funcionar. Si bien eran muy lentos en el diseño y armado, también tenía la contra de que si se quería reformar su lógica de funcionamiento, había que arrancar prácticamente desde cero. A este tipo de sistemas, se los denomina: "Automatización con lógicas de contactos".
Con la aparición de nuevos instrumentos y procesadores electrónicos, esas inmensas estructuras de relay, se fueron achicando y simplificando, hasta llegar a lo que hoy en día conocemos como: "Controladores Lógicos Programables" …

Por qué Arduino:

La selección de Arduino como núcleo de uPLC responde a una visión estratégica: convertir la complejidad industrial en accesibilidad universal. A diferencia de entornos cerrados o más complejos como CCS, PBP o ASM, Arduino ha derribado las barreras de entrada al mundo de los microcontroladores.

Elegimos esta plataforma por tres pilares clave:

- Escalabilidad y Costos: Hardware accesible que permite prototipar a bajo costo y escalar a soluciones industriales competitivas.
- Soporte Global: Una comunidad masiva y documentación inagotable que garantizan la sostenibilidad del proyecto a largo plazo.
- Reducción de la Brecha Digital: Es la herramienta que mejor comunica el mundo académico con el profesional.

Mi más sincero agradecimiento a Massimo Banzi, David Cuartielles, Tom Igoe, Gianluca Martino y David Mellis. Su visión no solo creó una placa, sino un lenguaje universal que hoy nos permite llevarte este interprete para la programación de un control lógico programable (PLC) a tu aula y cada escritorio mediante uPLC.

Este curso está diseñado para que aprendas desde CERO, aunque nunca hayas programado nada en tu vida.

Cada lección tiene:

- 📖 Teoría simple
- 💡 Ejemplo práctico
- ✏️ Ejercicio para hacer
- ✅ Solución

¡Empecemos!

INDICE

📖 **Lección n°1 Regla del Lenguaje LADDER**

📖 **Lección N°2 Elementos del Lenguaje**

📖 **Lección N°3 Operaciones Lógicas**

📖 **Lección N°4 Los Temporizadores**

📖 **Lección N°5 Entradas Análogas (Comparadores, Almacenar Variables)**

📖 **Lección N°6 Memorias y Lógicas de Retención**

📖 **Lección N°7 MODBUS - COMUNICACIÓN ENTRE PLCs**

📖 **Lección N°8 Funciones Matemáticas**

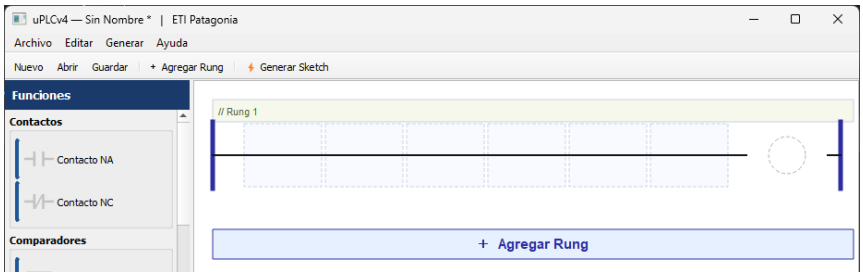
📖 **Lección N°8 Contadores**

📖 **Lección N°8 PID**

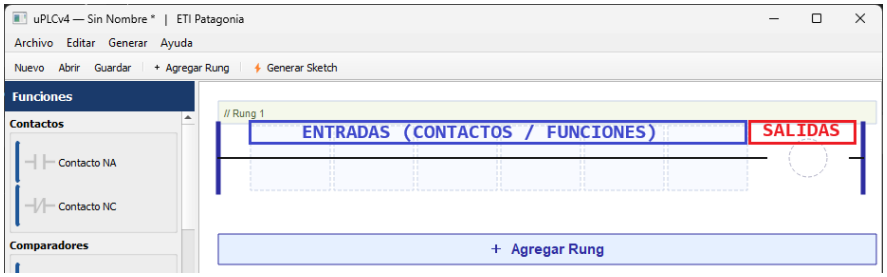
✏️ **Ejercicio FINAL**

🕮 Lección N°1 - Regla del Lenguaje LADDER

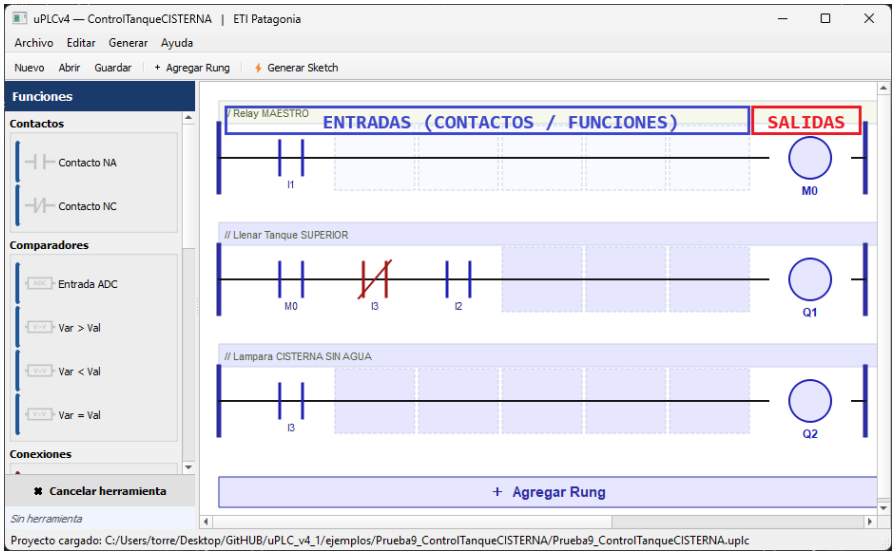
Siempre, el esquema de nuestra lógica, se realiza entre las dos líneas o barras azules de alimentación dispuestas verticalmente a ambos lados del diagrama, entre ellas se dibujan los elementos del lenguaje.



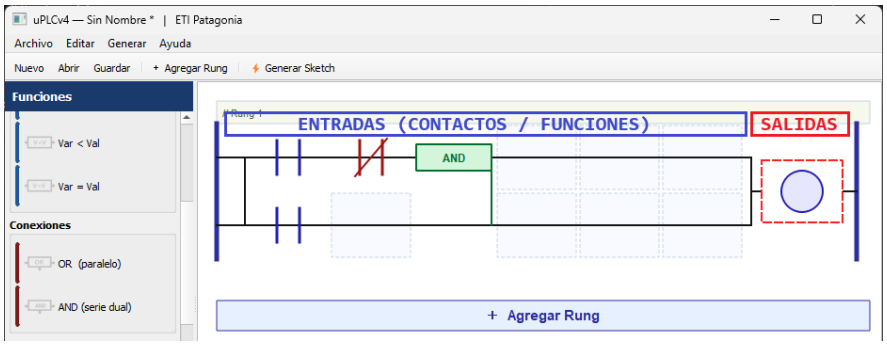
A la derecha del esquema se ubican los elementos de salida y a la izquierda los de entrada



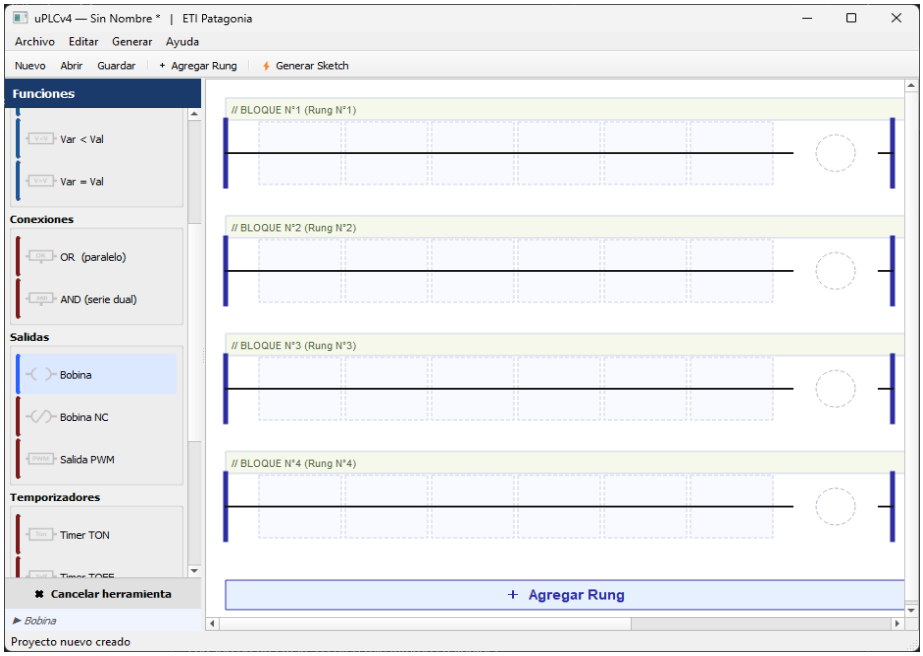
El diagrama puede tener varias ramas o escalones, llamados “BLOQUES o RUNG”.



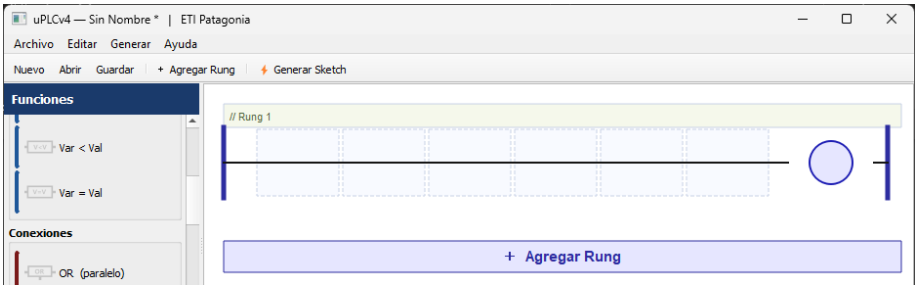
Cada rama permite ubicar varios elementos de entrada pero sólo uno de salida.



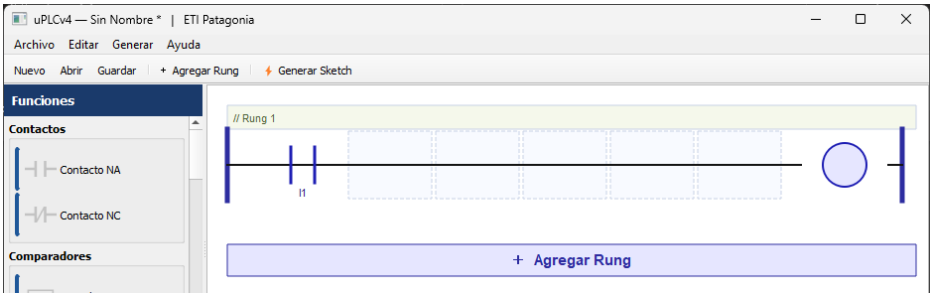
La programación en cada bloque de contactos se realiza en el orden de izquierda a derecha y el sentido de programación de los bloques de contactos de un programa de ejecuta en el sentido de arriba para abajo.



No se puede conectar una salida directamente a la línea principal…



en estos casos se intercala un contacto cerrado de una marca o bit o relé interno cualquiera.



Con relación a los contactos, tenga presente lo siguiente:

CONTACTOS DE ENTRADA	El número de contactos abiertos o cerrados que se pueden utilizar en un programa, por ca una de las entradas o espacios de memoria, es ilimitado; Es decir que se puede repetir el mismo número de contactos cuantas veces se quiera.
CONTACTOS DE SALIDA	El numero de salidas o bobinas de salida / relay de salida (OUT Físico) es fijo, por lo que n puede repetir un mismo numero de salida. Sin embargo, el numero de contactos asociado ellas es ilimitados. En el caso de las salidas o bobinas de memoria interna, no tiene limite y se pueden repetir varias veces en un programa

🕮 Lección N°2 - Elementos del Lenguaje

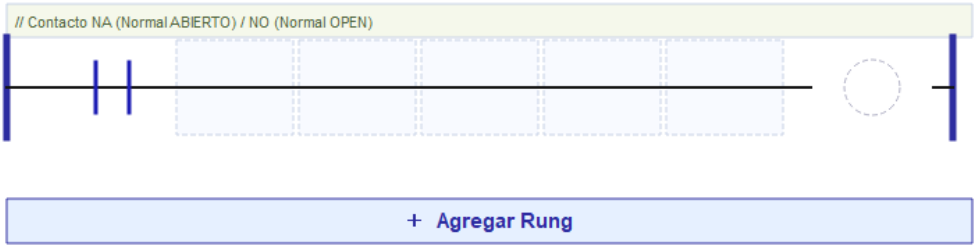
Se clasifican en elementos de entrada y salida. Su estado es evaluado por el PLC para determinar un valor lógico, que recibe distintas denominaciones dependiendo del contexto de trabajo.
A continuación, se presenta una tabla donde se relacionan las denominaciones de los contextos con las usadas en este curso (activo e inactivo).

VALORES LOGICOS		
CONTEXTO	**ACTIVO**	**INACTIVO**
Informática	True	False
	Verdadero	Falso
Algebra de Boole	V	F
	1	0
Electrónica Digital	High	Low
	H	L

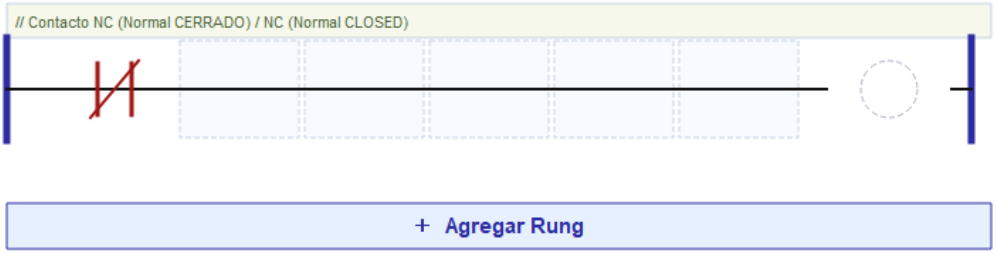
Elementos de Entrada

Los contactos, únicos elemento que se colocan a la entrada, son de tipo:

Normal Abierto (NO – Normal Open)



Normal Cerrado (NC- Normal Closed)



Debajo del contacto o función se selecciona la variable, espacio de memoria o puerto físico a la cual hace referencia. El valor lógico del contacto/función depende directamente del valor lógico de su variable o estado físico. Por ejemplo, para el contacto normal abierto, si la variable o estado físico es V, el contacto tendrá un valor de 1 lógico, y si la variable o estado físico es F, el contacto o función tendrá un valor de 0 lógico.
El contacto normal cerrado toman el valor inverso de su variable o estado físico, si la variable es V, el contacto tendrá un valor lógico “0”

🕮 Lección N°3 – Operaciones Lógicas

Las operaciones lógicas más utilizadas son: AND, OR, NOT… y veremos cómo podemos recrear la funcion EXOR (OR Inclusiva) jugando con las funciones y contactos que tenemos en uPLC.
A continuación, conoceremos las cuatro tablas de la verdad que las definen y en paralelo, iremos conociendo las funciones que tenemos disponibles…

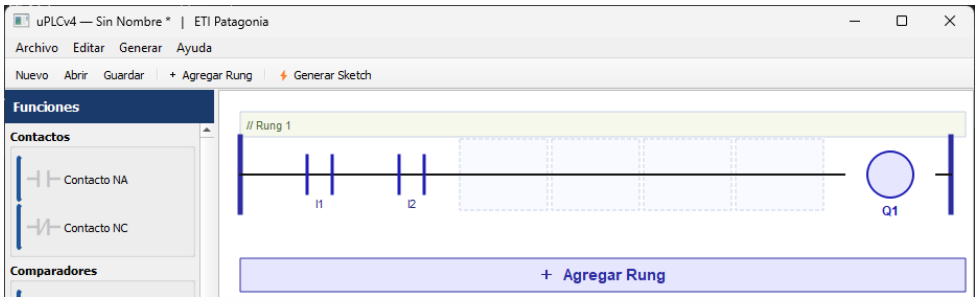
AND (Conjunción)

La operación lógica AND se aplica en situaciones en las que se requiere realizar una acción ***si*** y ***sólo sí*** se cumplen un determinado número de condiciones.

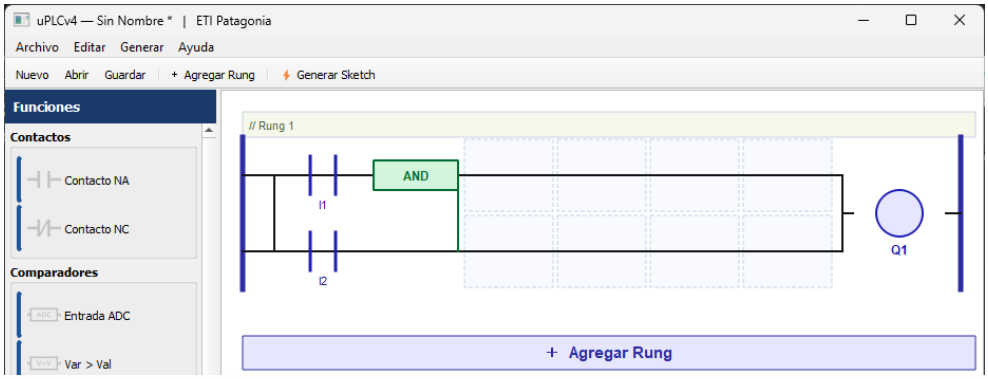
AND		
ENTRADAS		SALIDAS
A	B	Q
F	F	F
F	V	F
V	F	F
V	V	V

Veámoslo de la siguiente manera… A veces necesitamos que DOS o MÁS condiciones se cumplan JUNTAS; Por ejemplo: "Para entrar a mi casa necesito LLAVE **Y** CÓDIGO", Si falta alguno de los dos, no puedo entrar… Esto en Ladder lo podemos representar de las siguientes maneras:

Ponemos los contactos I1 e I2 UNO AL LADO DEL OTRO (en serie)



O bien, empleamos el bloque de función AND con sus contactos en cada entrada de función



💡 EJEMPLO: Máquina peligrosa con doble botón de seguridad: "*El motor solo arranca si presiono AMBOS botones a la vez*"

¿POR QUÉ DOS BOTONES?

Para que el operador tenga las dos manos ocupadas y no pueda meter las manos en la máquina mientras está funcionando.

🖊 **EJERCICIO 1 - Crea un programa donde necesitas p**resionar I1 (botón manual) y que I2 esté activado (sensor de seguridad), para que se encienda Q1 (motor)... Los pasos a seguir serian:

1. Nuevo proyecto
2. Agregar rung
3. Poner contacto NA en primera celda (I1)
4. Poner contacto NA en segunda celda (I2)
5. Poner bobina en salida (Q1)
6. Guardar como "Leccion1_AND.uplc"

....

7. Repite el mismo ejercicio, pero ahora emplea el bloque de “función AND”

✅ **SOLUCIÓN** – Al guardar o presionar el botón de “Generar SKETCH”, dentro de la carpeta “EJEMPLOS”, veremos que tanto de una manera como la otra, nos genero el programa completo y en el loop, el código generado será similar a este:

```
//==========================================================
// uPLCv4 — Sketch generado automáticamente
// ETI Patagonia - prof.martintorres@educ.ar
// Proyecto: Leccion1_AND
//==========================================================
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 20, 4); // Display LCD 20x4 I2C (dirección 0x27)
....definiciones de variables - etc...

void setup()
     {
      ....configuraciones...
     }

void loop()
    {
      ....resto del programa...
     // [1] Rung 1
    bool _r0 = ((I1) || (I2));
    Q1 = _r0;
   ....resto del programa...
   }
```

OR (Disyunción)

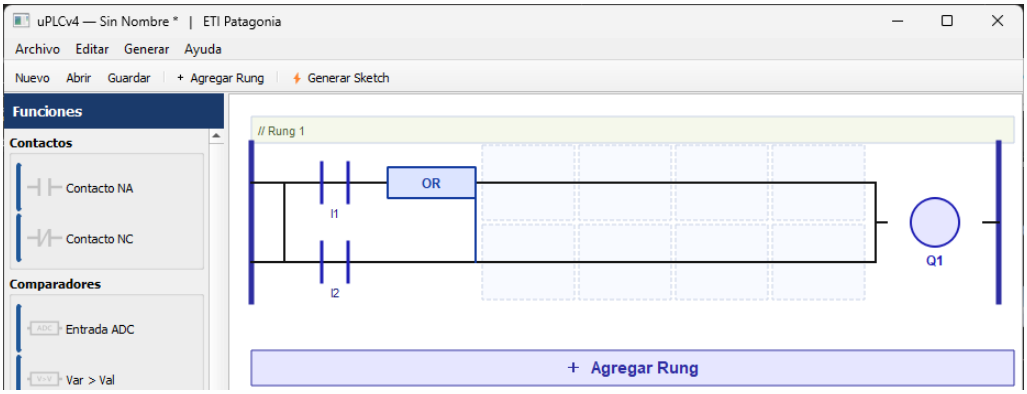
La operación lógica OR, entrega como resultado V (*VERDADERO*) siempre que alguna de las entradas sea V (*verdadera-presente-estado lógico uno-etc.*).

OR		
ENTRADAS		SALIDAS
A	B	Q
F	F	F
F	V	V
V	F	V
V	V	V

A veces queremos que con que UNA sola condición se cumpla, ya con eso alcance tengamos un resultado. Por ejemplo: “Puedo entrar a mi casa por la PUERTA DE ADELANTE “o” por la PUERTA DE ATRÁS”… Con que una esté abierta, puedo entrar.

Esto en Ladder lo podemos representar de la siguiente manera:

- Ponemos un contacto NA y seguido, sumamos la “Funcion OR”; En la otra entrada de la función, sumamos en paralelo otro contacto NA, y a la salida de la función, le asignamos una bobina de salida Q1



💡 EJEMPLO: Luz de pasillo con dos interruptores (uno arriba, uno abajo): "Desde arriba O desde abajo puedo encender la luz"…

🖊 EJERCICIO 2 - Creá un sistema de parada de emergencia

- Botón STOP 1 (I1)
- Botón STOP 2 (I2)
- Botón STOP 3 (I3)
- Cualquiera de los tres contactos NC para la máquina (apaga Q1)

PISTA: Usa contactos NC (Normalmente Cerrados) para los STOP; Implementa una salida con el nombre de M0 (*relay interno o espacio de memoria de uso interno*), un contacto NA (con el nombre de la memoria interna implementada) y una salida física que controla el motor (Q1)

Pasos en uPLC v4:

1. Crear rung.

2. Poner contacto NC I1.

3. Poner Bloque de Funcion OR.

4. Poner contacto NC I2 en la otra entrada del bloque de función.

5. Poner una Bobina de salida y nombrarla M0 (*relay interno o espacio de memoria interno*)

3. Clic el boton “Agregar Rung"

4. Poner contacto NC I3 y tenemos dos opciones, o sumamos función AND, o bien, sumamos en serie un contacto NA y lo nombramos (M0)

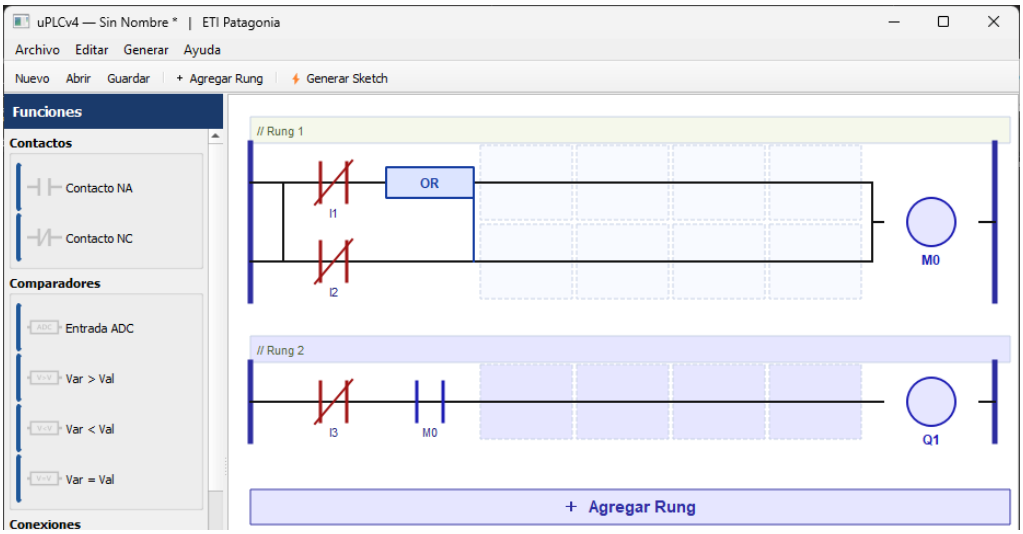
5. A continuacion de la serie AND, poner una Bobina y nombrarla Q1

¿POR QUÉ NC (Normalmente Cerrado)?

Porque queremos que la máquina FUNCIONE (Q1=1) cuando los botones “no” están presionados.

Si alguien presiona cualquier STOP → ese contacto NC se abre → se corta la corriente → Q1=0 → la máquina para.

✅ SOLUCIÓN



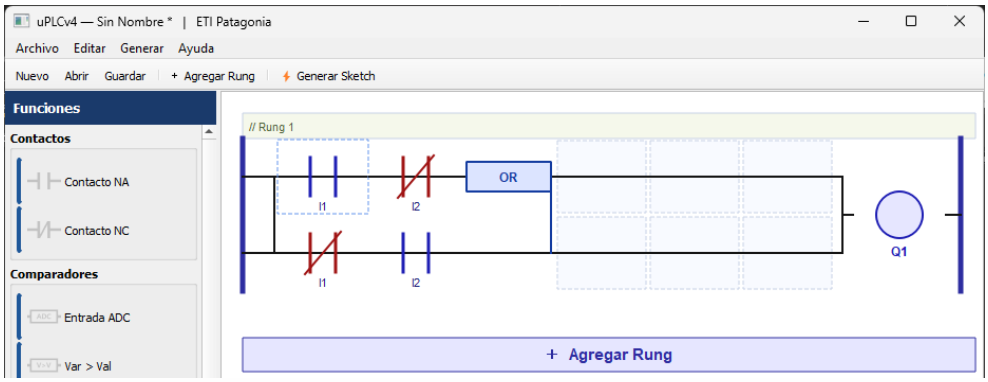
🎓 CONCEPTO IMPORTANTE: En sistemas de seguridad, SIEMPRE usar contactos NC para botones de emergencia. Así, si se corta un cable, la máquina para automáticamente (seguridad por defecto).

EXOR (OR Exclusiva)

La EXOR (OR exclusiva) es V (*VERDADERO*) si alguna de las entradas, pero nunca ambas, es V también; se puede decir que es V **si** y **sólo si** las entradas son distintas.

EXOR		
ENTRADAS		SALIDAS
A	B	Q
F	F	F
F	V	V
V	F	V
V	V	F

Esto en Ladder lo podemos representar de la siguiente manera:



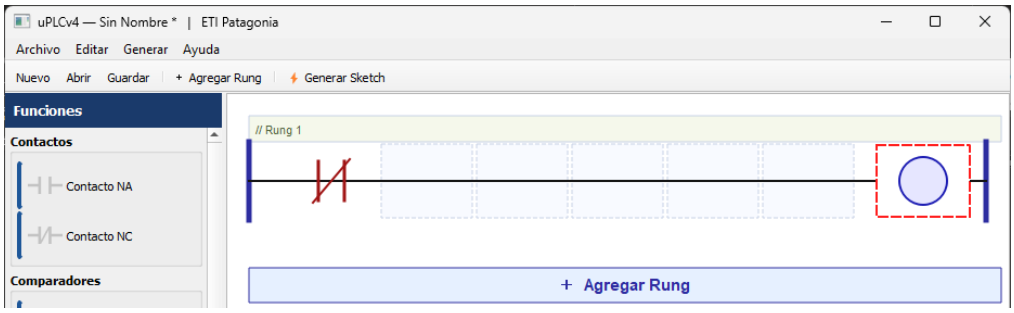
💡 EJEMPLO: "Controlar el sentido de giro de un motor, protegiendo de no tener contra orden y generar un cortocicuito"...

NOT (Inversión)

La operación lógica NOT, entrega como resultado el estado contrario al presente en la entrada

NOT	
ENTRADAS	
A	Q
F	V
V	F

Esto en Ladder lo podemos representar de la siguiente manera:



🕮 Lección N°4 - Temporizadores

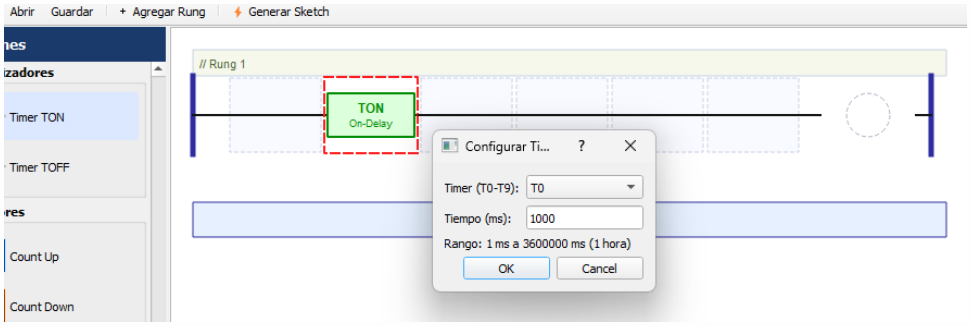
Los temporizadores son ESENCIALES en automatización.

HAY DOS TIPOS:

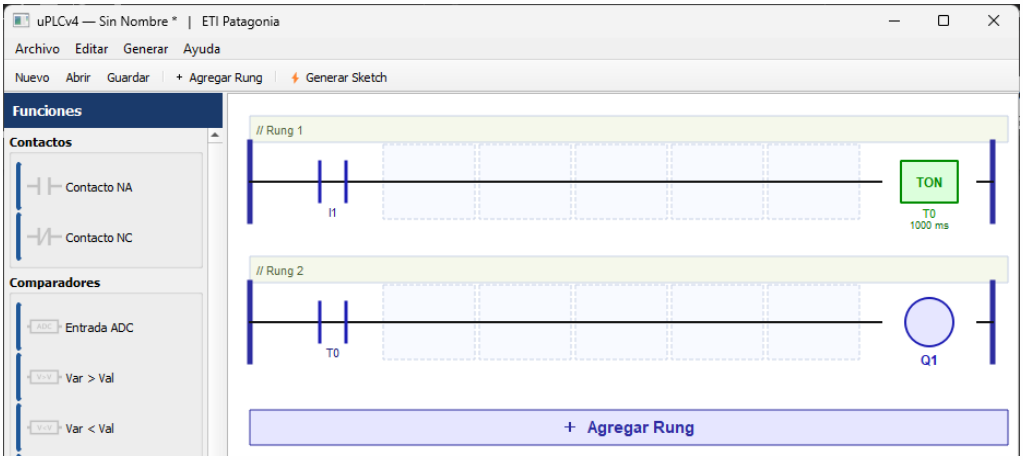
1. TON (Timer ON-delay): "Espera X tiempo DESPUÉS de activarse y se desactiva"

2. TOFF (Timer OFF-delay): "Cuando se activa, no funciona, pero cuando se desactiva, se mantiene activo X tiempo DESPUÉS de desactivarse"

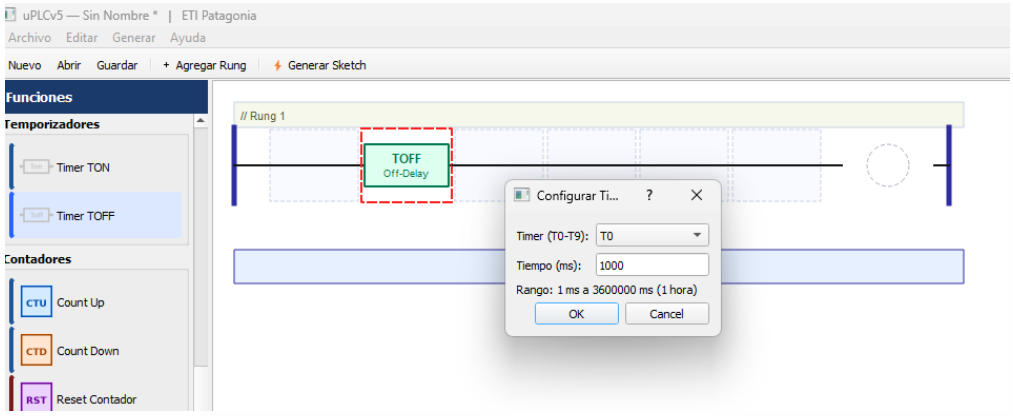
CÓMO USAR TON:



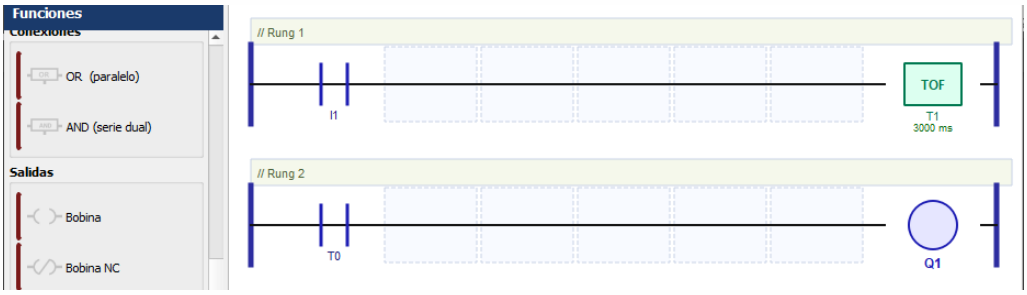
Presionamos el contacto NA “I1” y la salida Q1 se mantendrá encendida el tiempo configurado en el temporizador “T0” por el tiempo seleccionado (En este ejemplo 1000ms = 1seg)



CÓMO USAR TOF:



Presionamos el contacto NA “I1” y la salida se mantendrá apagada hasta que cambie de estado la entrada.... Al apagar “I1”, la salida Q1 se encenderá el tiempo configurado en el temporizador “T0” por el tiempo seleccionado (En este ejemplo 3000ms = 3seg)



💡 EJEMPLO: Cambio de sentido de giro de motor electrico

✏ Hasta este punto, creo que ya tenemos los conceptos más básicos e indispensables para poder crear **un sistema de cambio de sentido de giro de un motor t**omando las precauciones y seguridades para que nunca se puedan activar las ordenes de giro en un determinado sentido a la vez... ¿Te animas a sumar una parada de emergencia a este sistema donde I1 es la orden de giro derecho (activa el relay Q1) y I2 es la orden de giro izquierdo (activa EL RELAY Q2)?

Cuando hablo de precauciones y seguridades, apunto específicamente a tres puntos a tener en cuenta cunado trabajamos con motores inductivos ya sean monofásicos, como trifásicos.

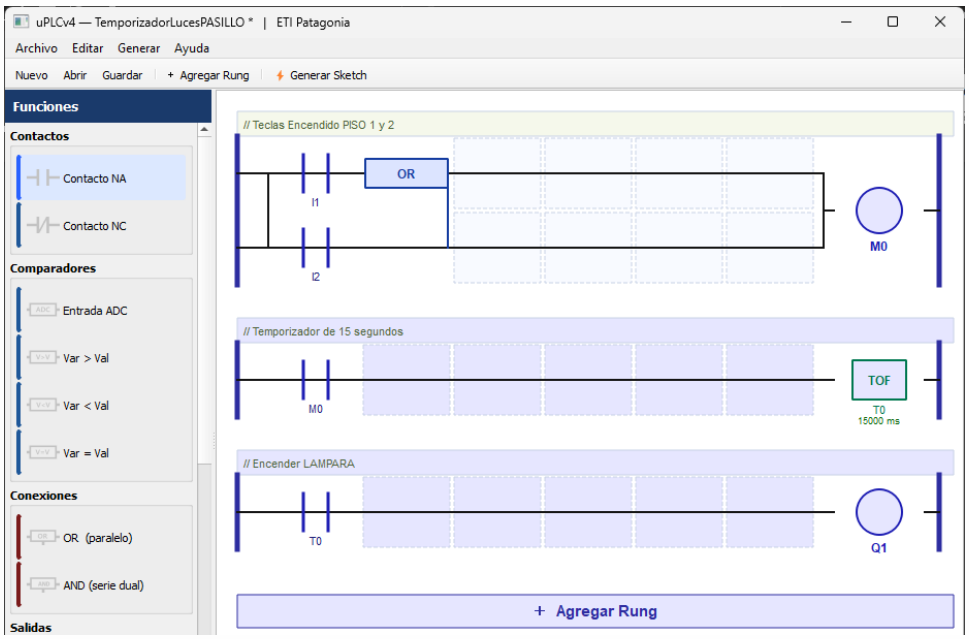
1_ prevención de Cortocircuito (Arco eléctrico). Cuando un contactor se desenergizar, sus contactos tardan milisegundos en abrirse. Además, al abrirse bajo carga, se genera un arco eléctrico entre los contactos; Si el segundo contactor entra mientras esta presente este arco eléctrico, provocara un Cortocircuito fase a fase, lo cual es una falla eléctrica grave.

2_ Fuerza Electromotriz residual (FEM). Incluso después de cortar la energía, el rotor del motor sigue girando y tiene un magnetismo remanente. Esto convierte al motor en un generador por un breve instante. Si conectas la alimentación en sentido opuesto mientras la FEM residual es alta, las corrientes pueden alcanzar valores de hasta 12 o 15 veces la corriente nominal, loque puede dañar los devanados o disparar las protecciones instantáneamente... En el mejor de los casos, puede que no se dispare las protecciones, pero el motor quizás sigue girando en el mismo sentido.

3_ Stress Mecánico. Cambiar de giro mientras el motor aun tiene mucha inercia hacia delante genera un choque mecánico violento en el eje, los rodamientos y las cajas reductoras. La pausa permite que la energía cinética disminuya, protegiendo la integridad física de la maquinaria.

💡 EJEMPLO: LUZ DE ESCALERA EN DOS PLANTAS.

"Presiono botón, luz enciende 15 segundos y se apaga sola" ¿Se te ocurre cómo podrías sumar mas pisos?



📖 Lección N°5 – Entradas Analogas

Hasta ahora vimos entradas DIGITALES (0 o 1, ON u OFF), TEMPORIZADORES, FUNCIONES BOOLEANAS, Pero muchos sensores dan valores VARIABLES. A este tipo de sensores, se los llama SENSORES ANALOGICOS.

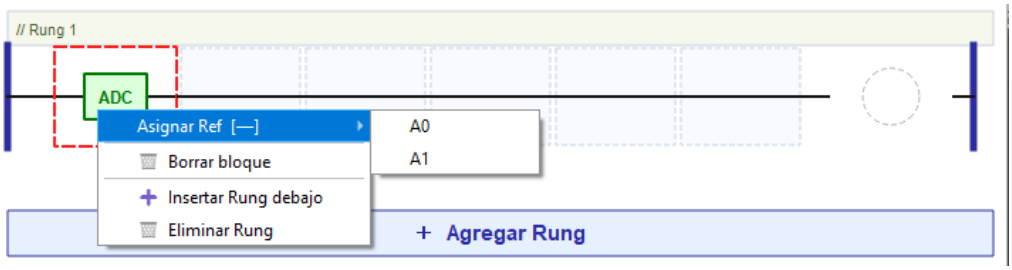
- Temperatura: 0°C a 100°C
- Luz: oscuro a brillante
- Distancia: cerca a lejos
- Presión: baja a alta

ENTRADAS ANALÓGICAS:

- A0 y A1 en nuestro Arduino
- Leen valores de 0 a 255

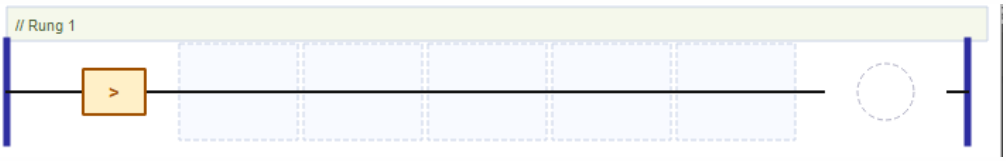
BLOQUES DISPONIBLES:

1. ADC (Analog-to-Digital Converter):

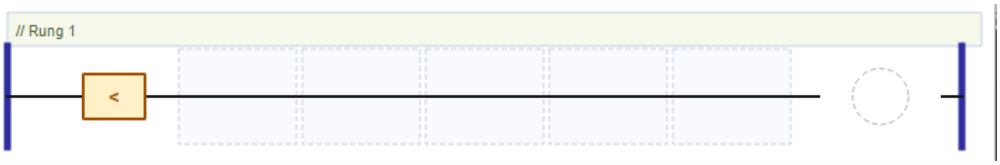


2. COMPARADORES:

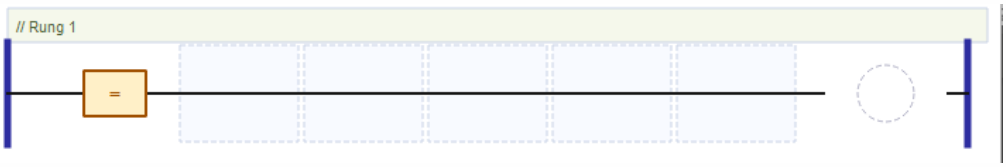
MAYOR QUE...



MENOR QUE...

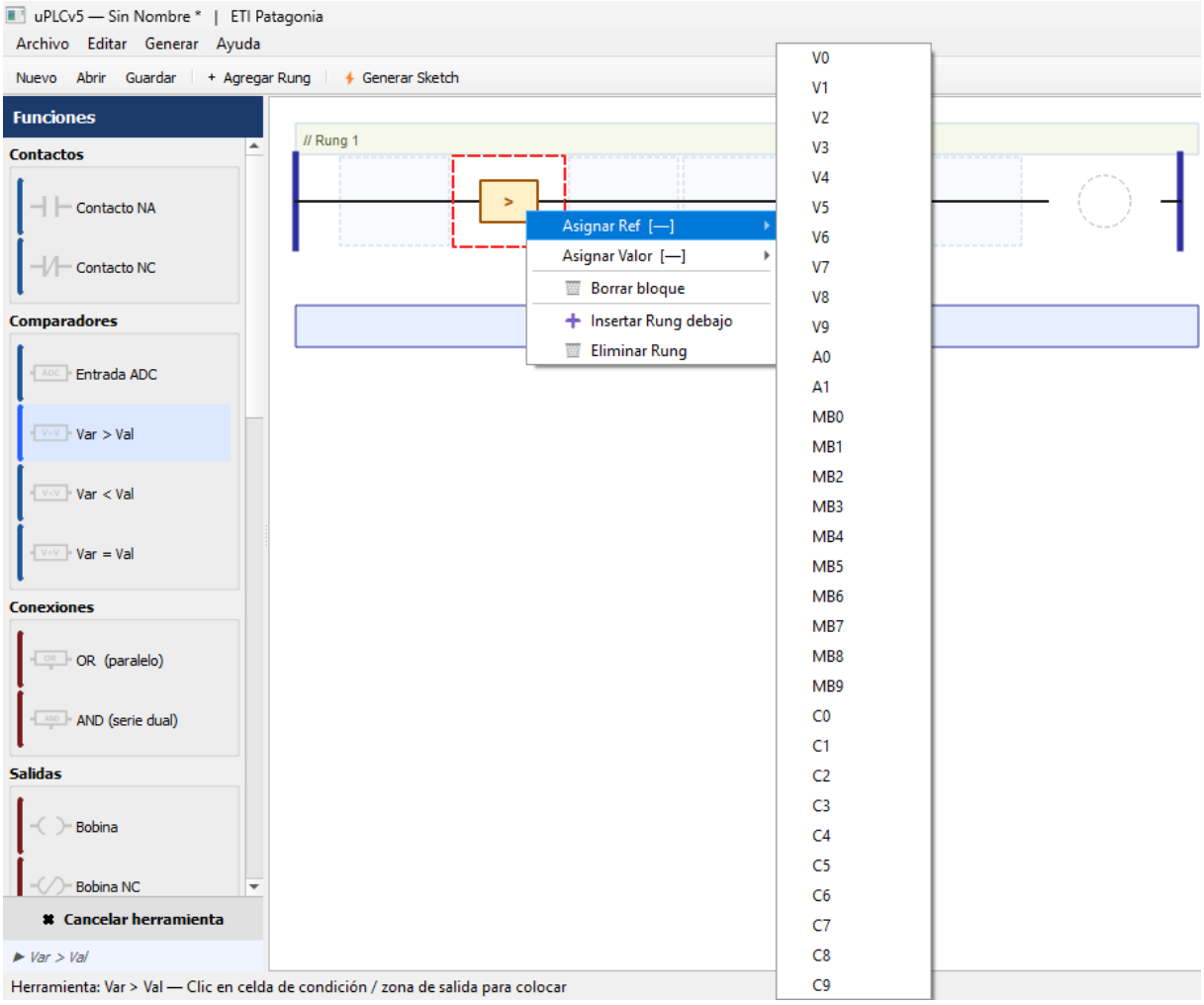


IGUAL QUE...

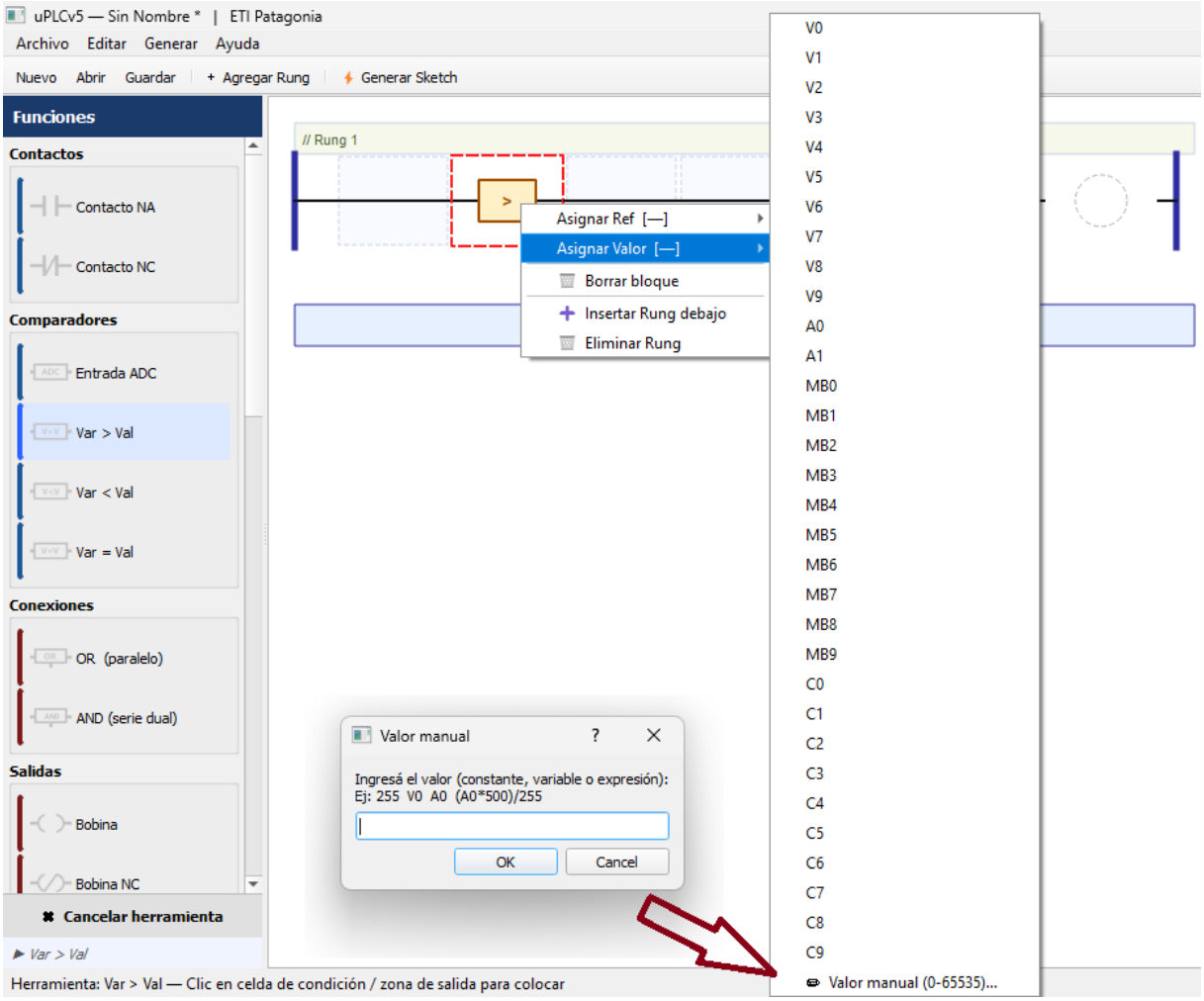


En cualquiera de los tres comparadores, siempre evaluamos 2 valores y en caso de confirmación, nos devuelve un TRUE y sigue el resto del bloque... Caso contrario, el programa queda en ese punto hasta que se cumpla la condición.

La configuración de los comparadores es sencilla. Primeros asignamos el primer valor a comparar, que puede ser una variable (V0 a V9), El valor leído directamente de cualquiera de los dos puertos analógicos (A0-A1) , o algún "Memory Bits" de una trama de comunicación Modbus, o el registro de algún contador

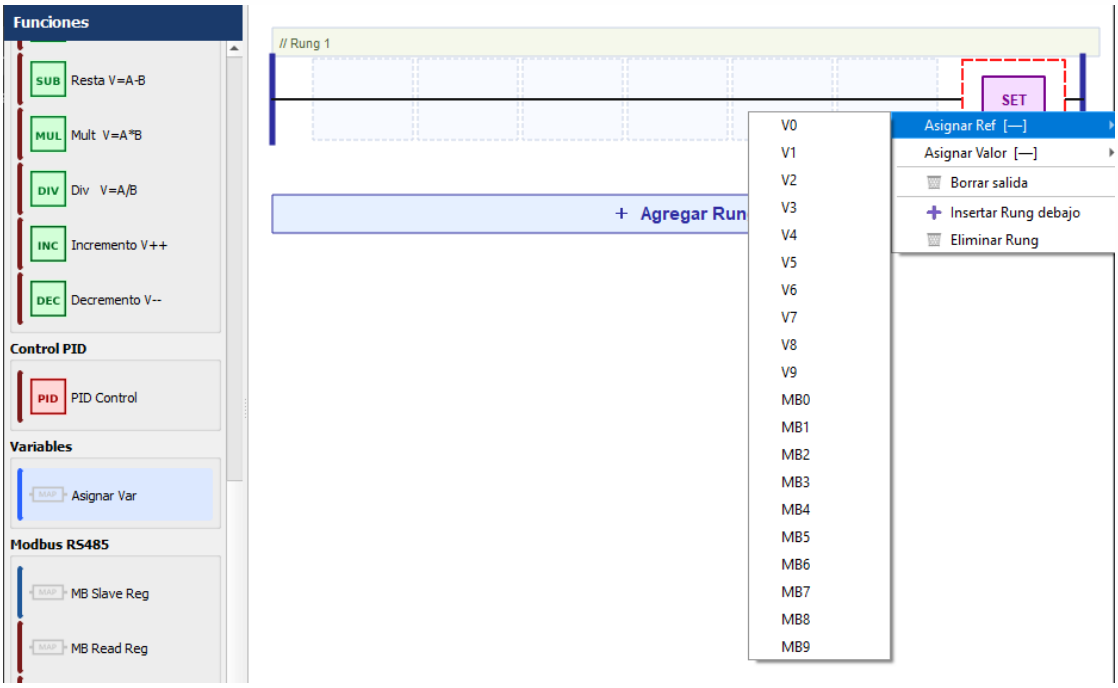


El segundo valor a comparar, puede ser una variable (V0 a V9), El valor leído directamente de cualquiera de los dos puertos analógicos (A0-A1) , o algun “Memori Bits” de una trama de comunicación Modbus, algún contador, o bien, podemos insertar manualmente un valor constante

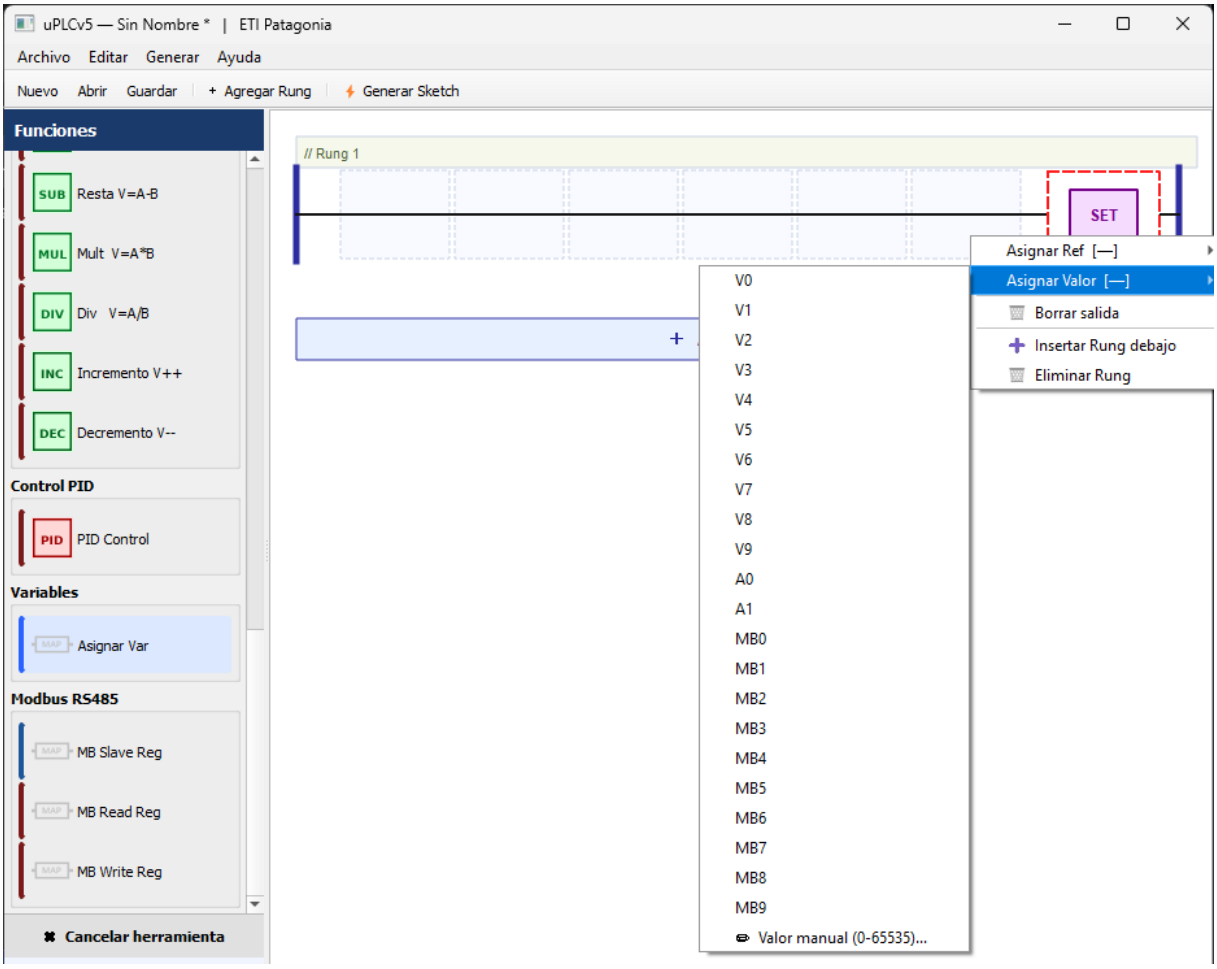


3. SET_VAR (Guardar valor):

Este bloque de función, tiene dos funciones especificas… Una es empleado para guardar determinados valores en un espacio de memoria (V0 a V9)…



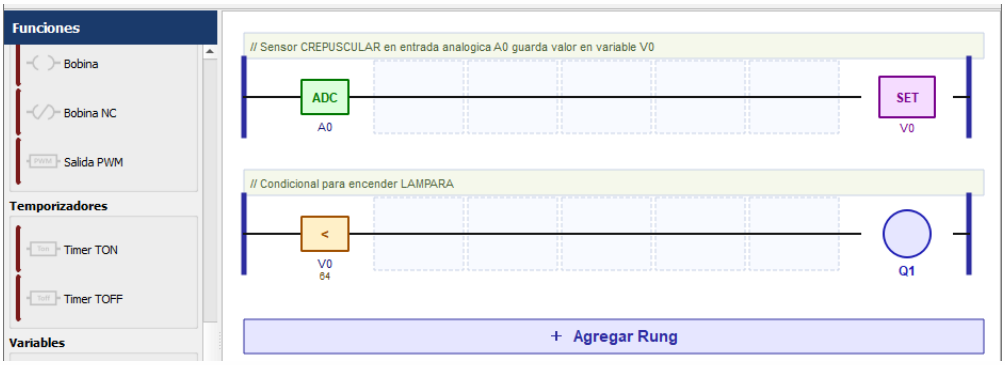
Y la otra función, es la de asignarle a un espacio de memoria una valor especifico, que puede ser otro espacio de variable, los valores de una entrada analógica, algún “Memory Bits” de una trama de comunicación Modbus, o bien asignarle una constante con el valor deseado para luego emplear la variable en otras operaciones del proyecto



💡 EJEMPLO: SENSOR CREPUSCULAR PARA ILUINACION: Quiero encender una luz cuando oscurece.

Sensor de luz en A0: 0 = oscuridad total / 255 = máxima luz

Programa: Si luz < 64, encender lámpara



¿POR QUÉ GUARDAR EN V0? Porque queremos usar el valor en múltiples lugares. Caso contrario, tendríamos que leer A0 varias veces.

🖉 **EJERCICIO** - Control de Recirculación de agua caliente en circuito de caldera: No tomemos en cuenta el proceso de calentar el agua, y solo vamos a enfocarnos y tomar como subproceso, el control de la bomba de recirculación de agua caliente por los circuitos de calefacción, que pasaran por los distintos elementos disipadores (radiadores) de la casa... Nuestro circuito tiene como condicion:

- Sensor temperatura en A0
- Potenciómetro en A1 para setear la velocidad de la bomba de agua de corriente continua
- Si temperatura < 12°C → Recirculación OFF (Q1)
- Si temperatura > 30°C → Recirculación OFF (Q1)
- Fuera 12-31°C → Todo apagado

Conversión: Asumimos que:

- A0 = 0 → 0°C
- A0 = 255 → 100°C
- Entonces: 12.5°C ≈ 32 / 31.37°C ≈ 80

✅ SOLUCIÓN

🎓 CONCEPTO: HISTÉRESIS

PROBLEMA: Si la temperatura está en 30°C exacto, el ventilador se prenderá y apagará constantemente (rebote).

SOLUCIÓN: Usar dos umbrales de la siguiente manera

- Encender ventilador si > 31°C
- Apagar ventilador si < 29°C

Esto se llama HISTÉRESIS y evita oscilaciones.

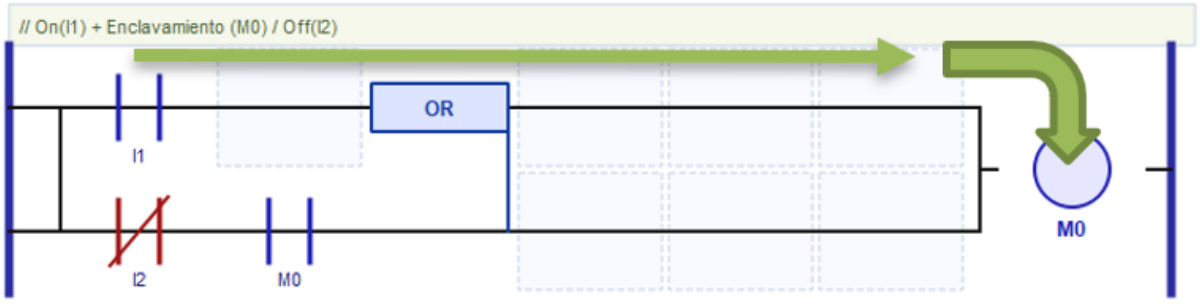
📖 Lección N°6 – Memorias y Logica de Retencion

A veces necesitamos que algo "se quede" encendido incluso después de soltar el botón, requerimos guardar ese estado en un espacio de memoria. Este estado que en algún momento paso de un estado a otro, tiene un valor booleano (Verdadero/falso/1 o 0).

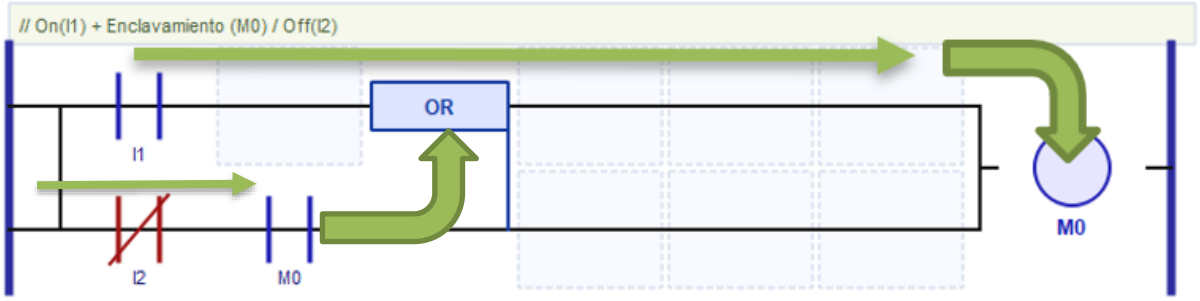
Esas memorias guardadas (también llamadas marcas / banderas, etc), pueden servirnos para implementarla en cualquier parte del programa, y muchas veces, es empleada para enclavar una acción hasta que el valor de este espacio cambie...

El ejemplo mas sencillo, es el enclavamiento de una bobina de salida, por medio de una memoria. Imaginemos pulsar momentáneamente un pulsador afectado a una entrada (I1)... Anteriormente vimos

ejemplos que la salida solo se accionaba cuando esta entrada estaba en estado alto; Pero que pasa si en vez de interactuar directamente con la salida, guardamos esa acción en un estado de memoria y realimentamos con una función OR esa salida con un contacto que tenga esa información....

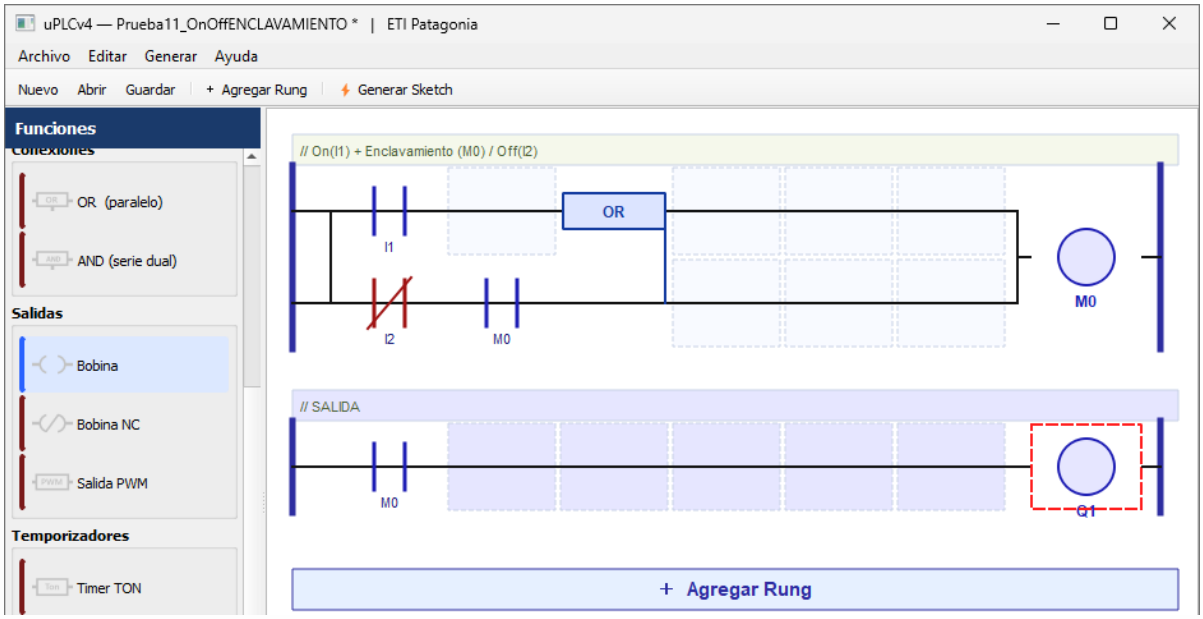


Bien , lo que pasaría es que en el contacto NA nombrado como "M0" realimentaría por intermedio de la función OR la misma salida que la enciende y se genera un loop continuo hasta que se interrumpa ese flujo



O sea, para APAGAR, si presiono la entrada "I2" interrumpo ese flujo y rompo el enclavamiento; O sea, apago el circuito.

El circuito completo en LADDER seria el siguiente:



🕮 Lección N°7 – Modbus – Comunicación entre PLCs

Modbus permite que varios dispositivos "hablen" entre sí… Veamoslo de esta manera: La comunicación mediante este protocolo, es como un grupo de WhatsApp de Arduinos. Uno es el JEFE (Master) que hace preguntas, y los demás son EMPLEADOS (Slaves) que “solo” responden enviando informacion de su estado, o bien, accionando ordenes impartidas por el jefe (encender / apagar salidas, etc).

Digamos que los roles están bien definidos y ninguno hará otra cosa extra:

- MASTER (Arduino 1):

  • Hace preguntas: "¿Cuál es tu temperatura?"

  • Envía órdenes: "Enciende tu ventilador"

  • Controla la comunicación

- SLAVES (Arduino 2, 3, 4...):

  • Esperan preguntas

  • Responden con datos

  • Obedecen órdenes

  • Cada uno tiene un ID único (2, 3, 4...) y es la manera de saber a quien hablarle

- REGISTROS: Son "cajitas" donde se guardan información en formato de 8 bits (0-255). Cada Arduino tiene 10: MB0, MB1, ... MB9 y cada cajita, tiene una función especifica…

💡 EJEMPLO: SISTEMA DE CLIMA

Setup:

  • MASTER: Controla ventilador

  • SLAVE ID=2: Lee temperatura living

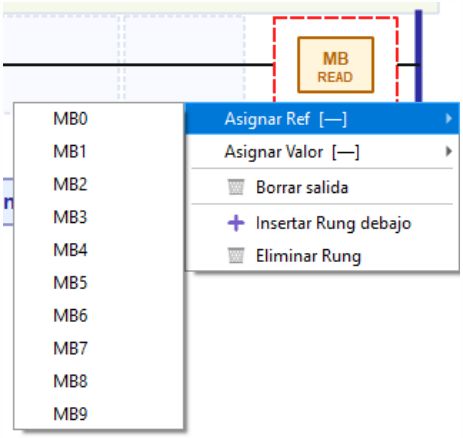
  • SLAVE ID=3: Lee temperatura dormitorio

Programa MASTER:

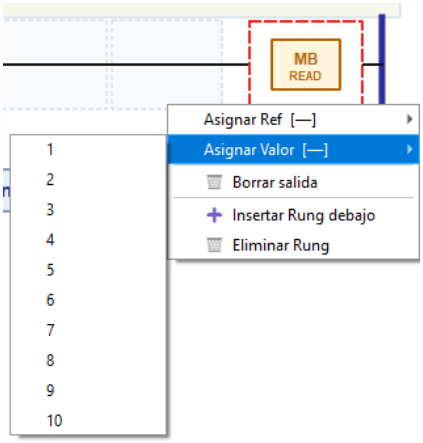
BLOQUES MODBUS:

1. MB_READ (Leer de Slave):

MB0 ← Registro local donde guardar

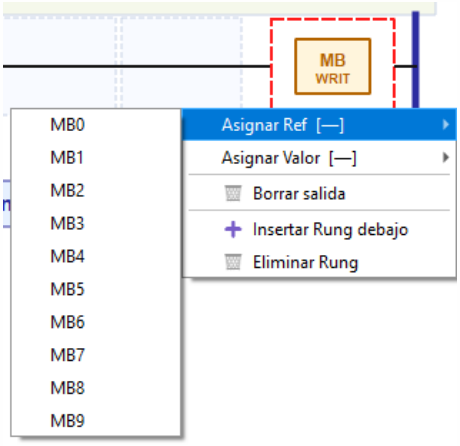


S:2 ← Slave ID=2

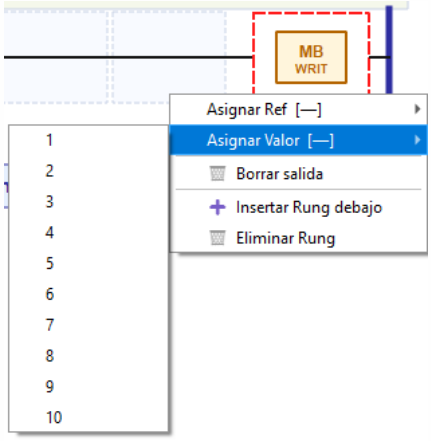


2. MB_WRITE (Escribir a Slave):

V0 ← Valor a enviar

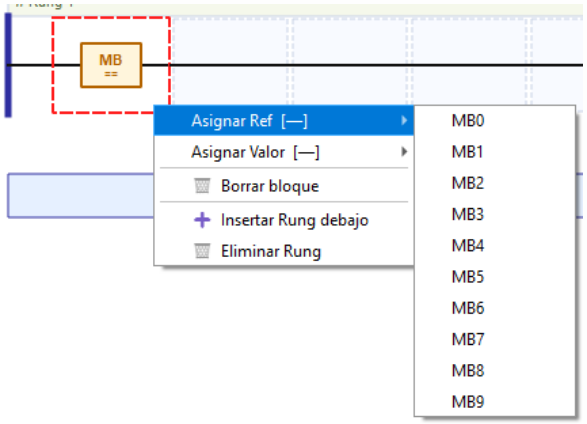


S:3 ← Slave ID=3

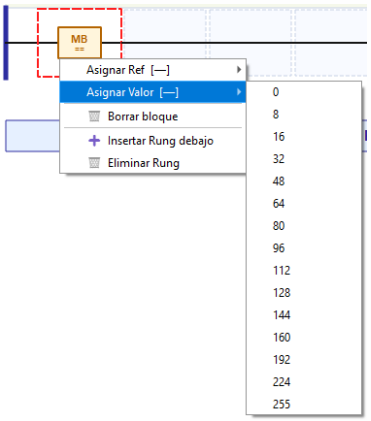


3. MB_SLAVE (Condición sobre registro):

MB0 ← Registro



0a255 ← Valor comparación

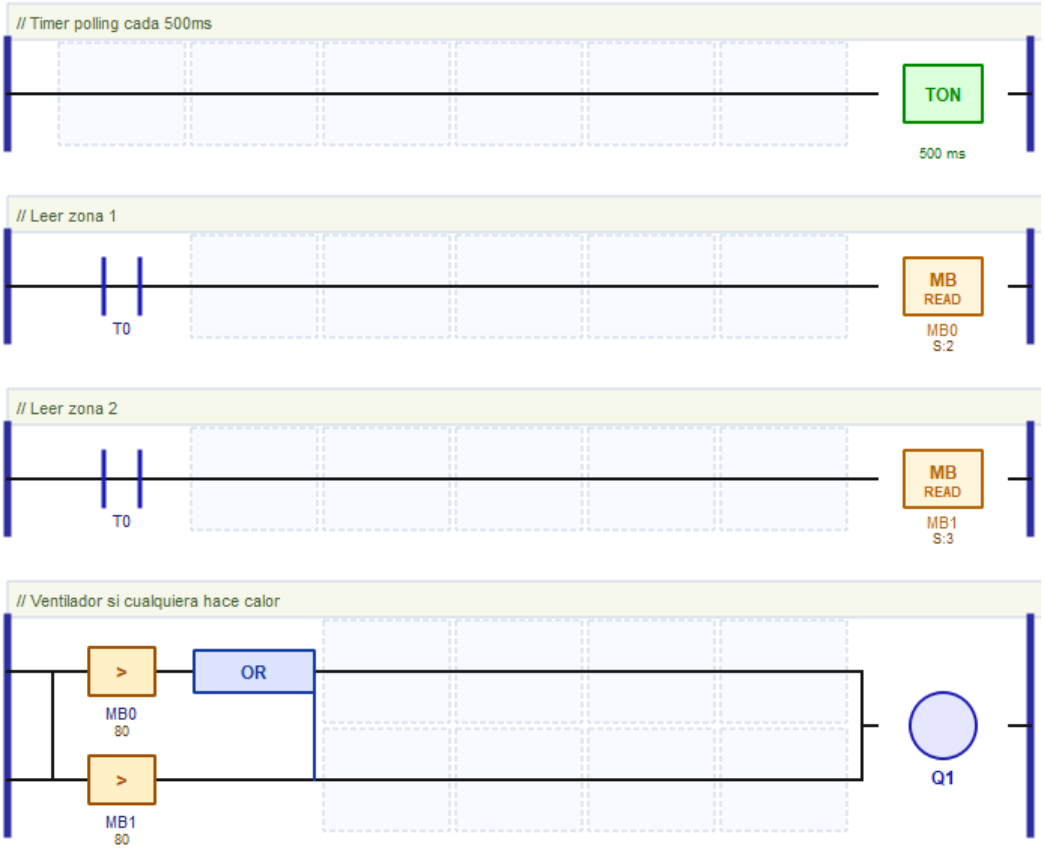


🖊 EJERCICIO 9.1

Sistema multizona:

- MASTER lee temperatura de 2 slaves
- SLAVE 2: Temperatura zona 1
- SLAVE 3: Temperatura zona 2
- Si CUALQUIER zona > 30°C → Ventilador general ON

✅ SOLUCIÓN - Programa MASTER:



🎓 Para que esto funcione necesitas, requieres el siguiente Hardware:

- 3 Arduinos NANO
- 3 Módulos RS485 (MAX485)
- Cable par trenzado
- 1 resistencia 120Ω (terminación)

Conexiones:

- A ←→ A (todos en paralelo)
- B ←→ B (todos en paralelo)
- GND común
- 120Ω entre A-B en el primero y último

🕮 Lección N°8 – Funciones Matemáticas

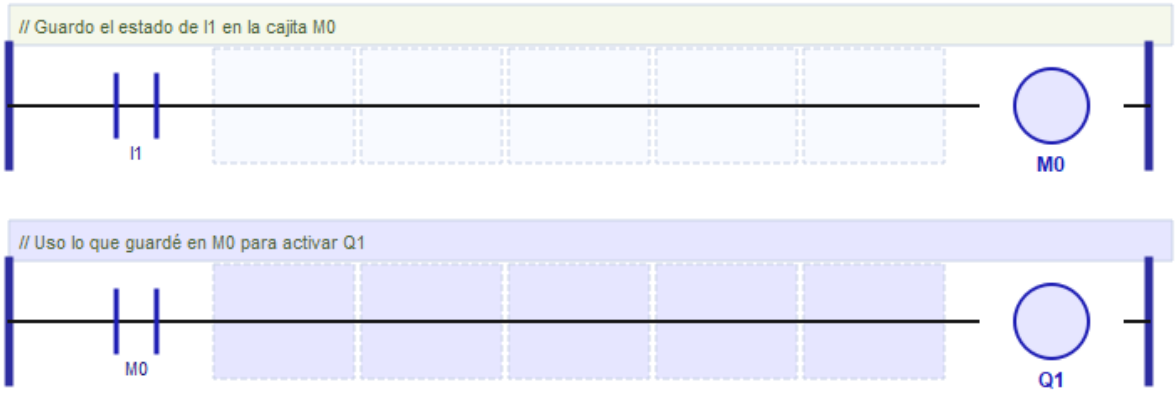
Antes de adentrarnos en lo que son los bloques de funciones matemáticas, primero tenemos que entender un par de conceptos y funcionalidades de uPLC.
La programación sería prácticamente imposible sin el uso de variables. Para poder entenderlas, intentemos imaginar a una variable como si fuera una caja donde guardamos cosas...
Esa caja es una de las muchas que disponemos, y tiene en su frente pegada una etiqueta con su nombre. Estas cajas tienen ciertas particularidades, que hace que solo se puedan guardar en ellas determinados tipos de objetos.
En esta analogía, cada caja es una variable, su contenido es el valor que adopta, y la etiqueta es el nombre de la variable. Como su nombre indica, el contenido de una variable puede ser modificado a lo largo del programa mediante las operaciones matemáticas y los bloques de asignación.

Tipos de "Cajas" (Variables) en uPLC

En uPLC tenemos distintos tipos de variables, según el dato que puedan almacenar:

1_ M0 a M9 - Memorias (Flags o Banderas): Son como cajitas muy pequeñas que solo pueden guardar dos cosas: una ficha ROJA o una ficha VERDE. Nada más.
Capacidad: 1 bit de longitud
Contenido: Solo puede ser 0 (APAGADO/FALSE) o 1 (ENCENDIDO/TRUE)
Uso típico: Guardar estados tipo "¿está encendido?" o "¿se cumplió la condición?"

Ejemplo en ladder:



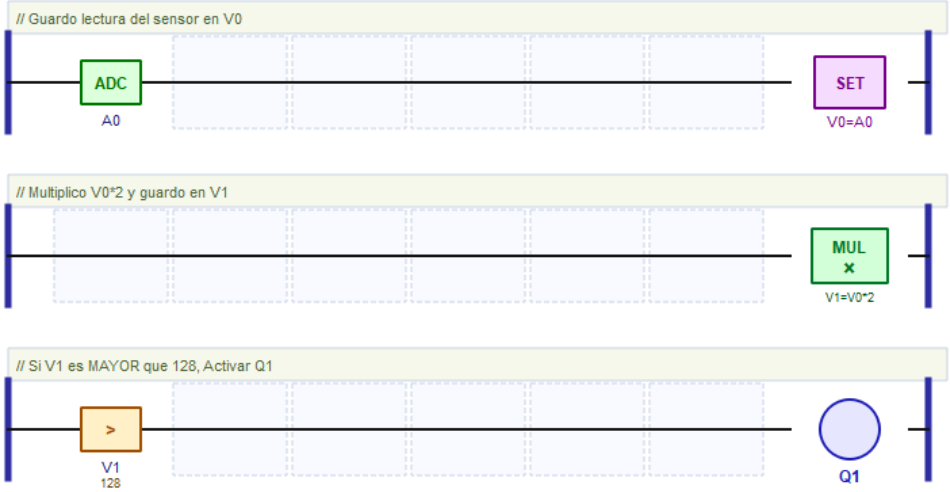
Imaginate que M0 es una cajita con dos compartimentos: uno dice "SÍ" y otro "NO". Solo podés poner una bolita en uno de los dos.

2_ V0 a V9 - Variables de 8 bits: Son cajas medianas que pueden guardar números más grandes.
Capacidad: 1 byte (8 bits)
Contenido: Números enteros de 0 a 255
Uso típico: Guardar lecturas de sensores, resultados de cálculos, contadores

¿Qué significa 8 bits?
Un byte está compuesto por 8 bits, y se lee de derecha a izquierda: **[bit.7 - bit.6 - bit.5 - bit.4 - bit.3 - bit.2 - bit.1 - bit.0]**
Por ejemplo:
11111111 = 255 (el número más grande que cabe)
00000000 = 0 (vacío)
00001010 = 10 (el número 10)

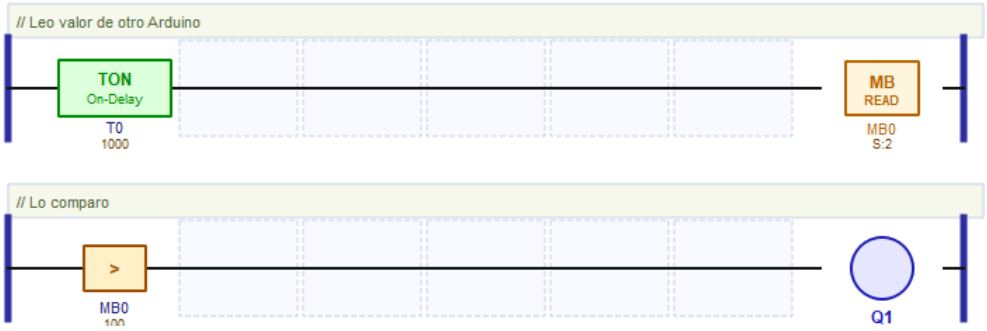
Imaginate que esta caja tiene 8 lucecitas. Cada lucecita puede estar prendida (1) o apagada (0). Dependiendo de cuáles estén prendidas, el número que representa cambia.

Ejemplo en ladder:

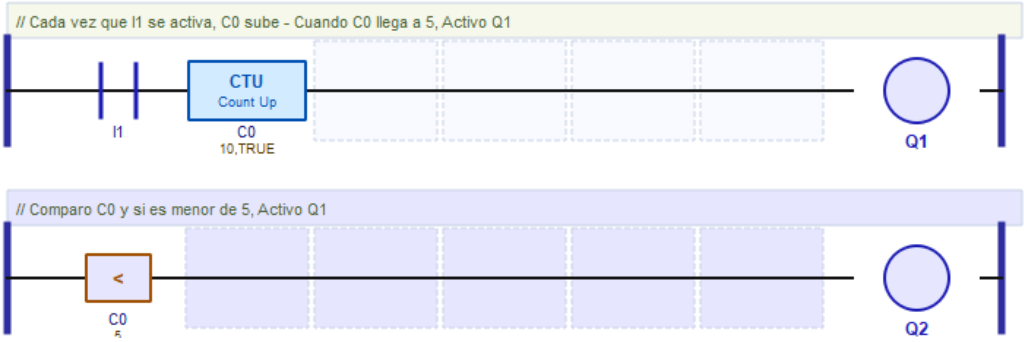


3_ MB0 a MB9 - Registros Modbus de 16 bits: Son cajas grandes que pueden guardar números mucho más grandes.
Capacidad: 2 bytes (16 bits)
Contenido: Números enteros de 0 a 65,535
Uso típico: Comunicación con otros Arduinos por Modbus, valores grandes

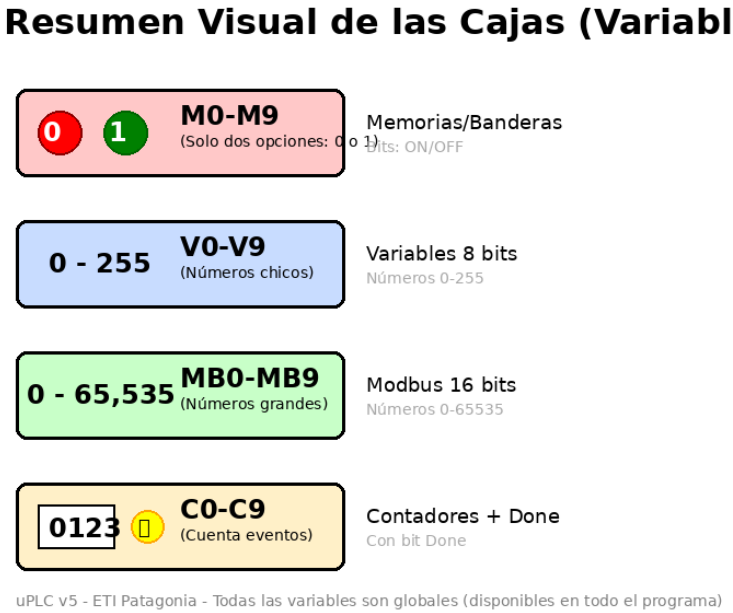
Imaginate que estas cajas son el doble de grandes que las V. Por eso pueden guardar números más grandes (hasta 65 mil y pico).
Ejemplo en ladder:



4_ C0 a C9 – Contadores: Estos son especiales. Son como cajas con un contador mecánico adentro (como los que cuentan las personas que entran a un local).
Contenido: Un número que va subiendo (CTU) o bajando (CTD)
Extra: Tienen una lucecita (Done bit) que se enciende cuando llegan al objetivo
Uso típico: Contar piezas, ciclos, eventos
Ejemplo en ladder:



Imaginate que C0 es una caja con un numerito que va cambiando. Cuando llega a 5, RING! suena una campanita (el Done bit se activa).

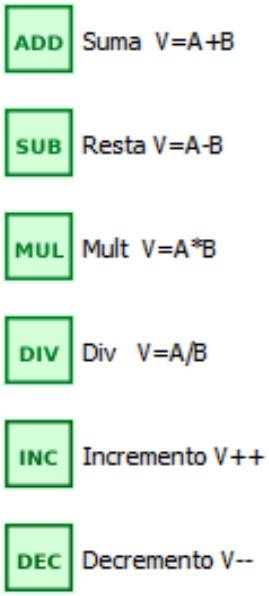


A diferencia de otros lenguajes, en uPLC no necesitás declarar las variables antes de usarlas. Simplemente las usás cuando las necesitás:

Las 10 memorias M0 a M9 siempre están disponibles
Las 10 variables V0 a V9 siempre están disponibles
Los 10 registros Modbus MB0 a MB9 siempre están disponibles
Los 10 contadores C0 a C9 siempre están disponibles

Todas son globales, es decir, podés usarlas en cualquier rung del programa. Si guardás algo en V0 en el rung 1, podés leerlo en el rung que quieras, por más que ya estes trabajando en el rung 55, 90, etc.

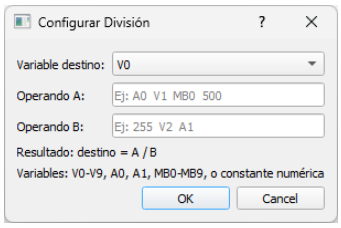
Bien… Creo que con estos conceptos, ya podemos ir conociendo los distintos bloques de funciones Matemáticas…

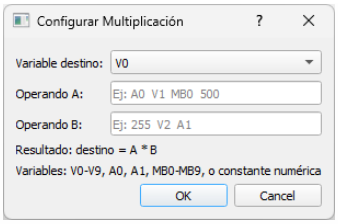


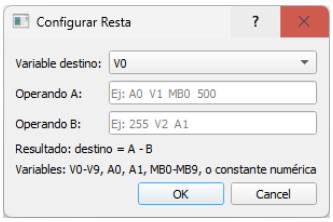
En la gran mayoría de las operaciones matemáticas, tenemos en común que siempre tendremos que configurar algunos datos para su empleo o llamada en el resto del programa.

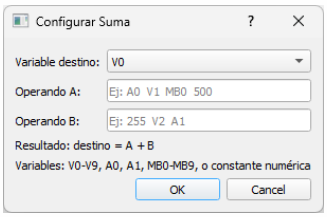
La primera configuración obligatoria, es definir el destino del resultado; O sea, donde guardaremos este dato. La segunda operación será seleccionar el operando A, que puede ser una variable, constante o bien, el estado de un puerto analógico. La tercera acción, será configurar el operando B, que al igual que el otro operando, deberá ser una variable, constante o estado de un puerto analógico.

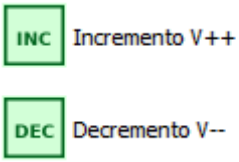
Tengan en cuenta que muchas veces el resultado puede ser muy grande para guardarla en una cajita de 8 bits, entonces, tengamos la precaución de verificar que los valores esten dentro de los valores permitidos para cada acción (Por ejemplo, el PWM solo puede trabajar con un valor asignado de 0 a 255)…





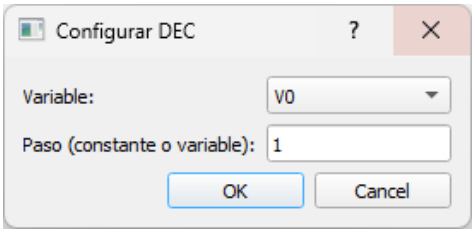


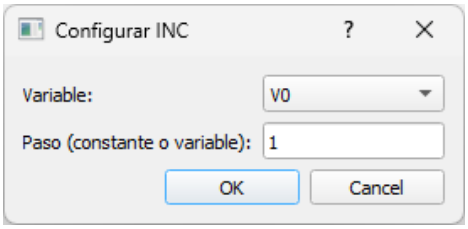




Los bloques de función "Incrementar/Decrementar VARIABLE", funcionan similar a los bloques de función "Contador UP/DOWN" que veremos mas adelante, y la diferencia radica en que este valor se asigna a una variable V0 a V9 y no tiene limite de conteo, ni estado de salida; Solo hace la operación y guarda en la variable.

La configuración de este bloque, solo requiere que asignemos la variable a trabajar, y cuanto queremos incrementar o decrementar esta variable cuando recibimos la orden o llamado a este bloque de función. Este bloque cuenta por seguridad de falsos conteos con detección de cambios de estados de flanco 0→1 / 1→0 (Si mantenés el botón / pulsador o la orden de otro bloque se mantiene en alto, solo incrementa "UNA" vez); Y también cuenta con una protección anti desborde que no decrementa por debajo de cero.





🕮 Lección N°9 – Contadores

Los contadores permiten contar eventos como piezas producidas, ciclos de máquina, o pulsos de sensores.

IMPORTANTE - Detección de Flanco:

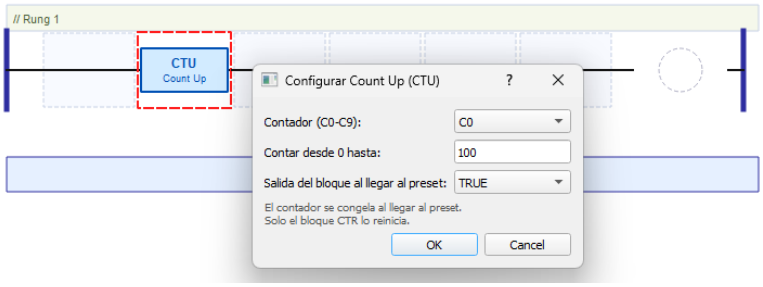
Los contadores cuentan solo en la transición 0→1 (flanco ascendente).
Si mantenés la entrada presionada, solo cuenta UNA vez... Esto evita conteos falsos.
En uPLC, tenemos tres bloques de funcion en la sección contadores:

CTU - Contador Ascendente

CTD - Contador Descendente

CTR - Reset de Contador

1_ EL CONTADOR ASCENDENTE: Cuenta hacia arriba desde 0 hasta un valor objetivo (Preset Value).



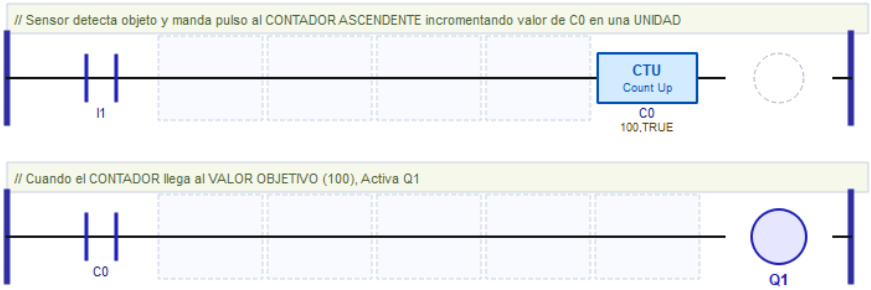
Configuración:

- Ref: C0 a C9 (qué contador usar)
- PV (Preset Value): Valor objetivo (ej: 100 piezas)

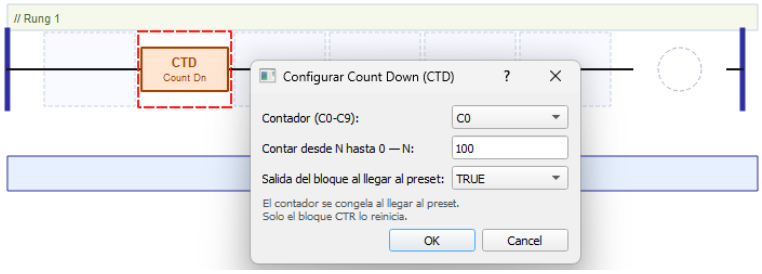
Bits disponibles:

- C0.Q (Done bit): TRUE cuando CV >= PV
- CV (Current Value): Valor actual del contador

Ejemplo en ladder:



2_ EL CONTADOR DESCENDENTE: Cuenta hacia abajo desde un valor inicial hasta 0.



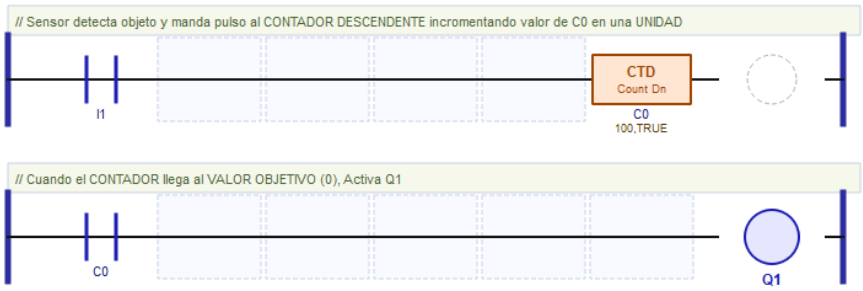
Configuración:

- Ref: C0 a C9
- PV: Valor inicial (ej: 50)

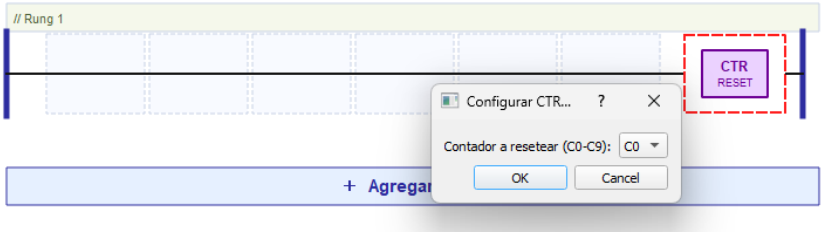
Bits disponibles:

- C0.Q (Done bit): TRUE cuando CV = 0
- CV: Valor actual

Ejemplo en ladder:



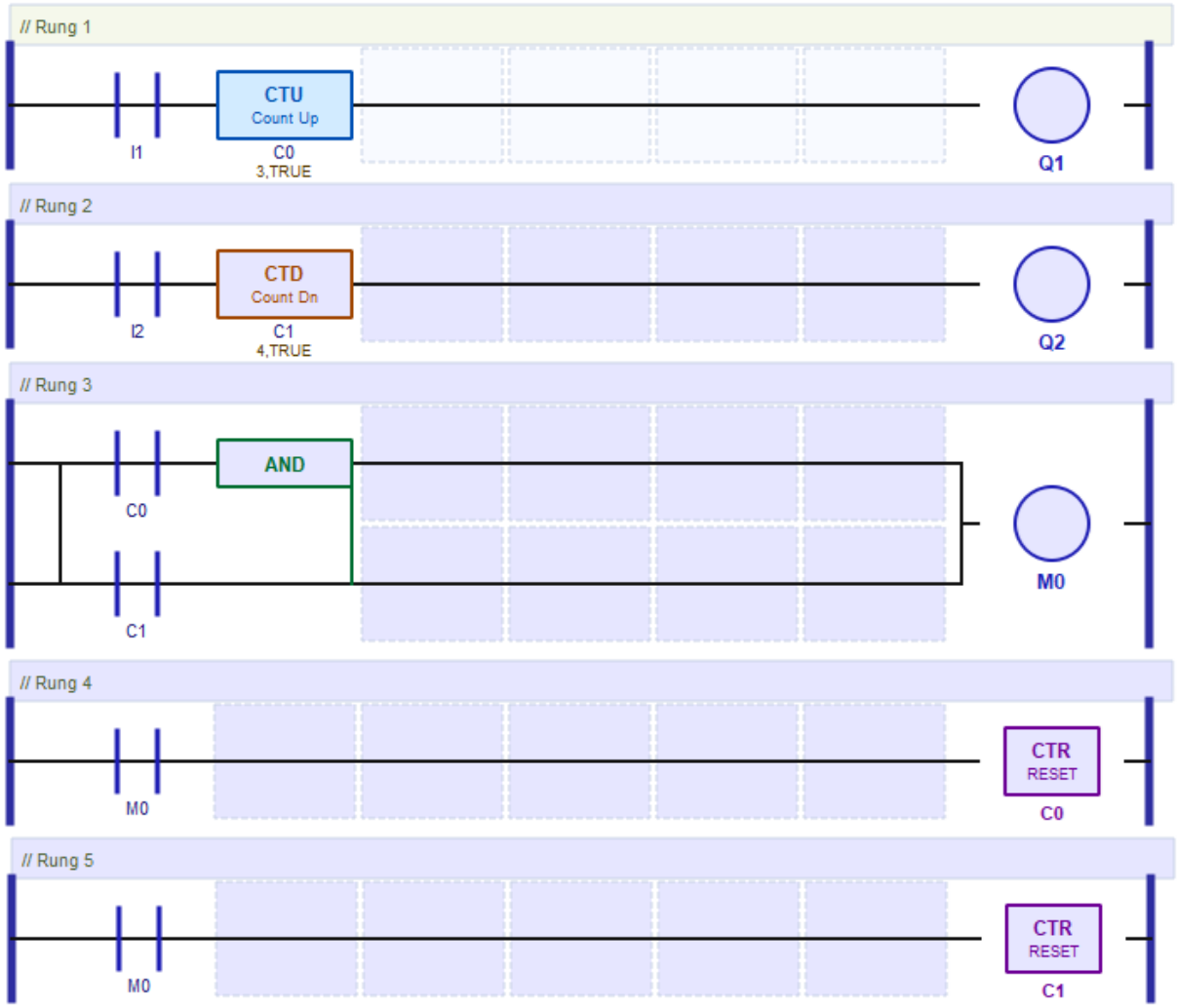
3_ EL RESET DE CONTADOR: Reinicia un contador a cero.



Configuración:
- Ref: C0 a C9 (qué contador resetear)

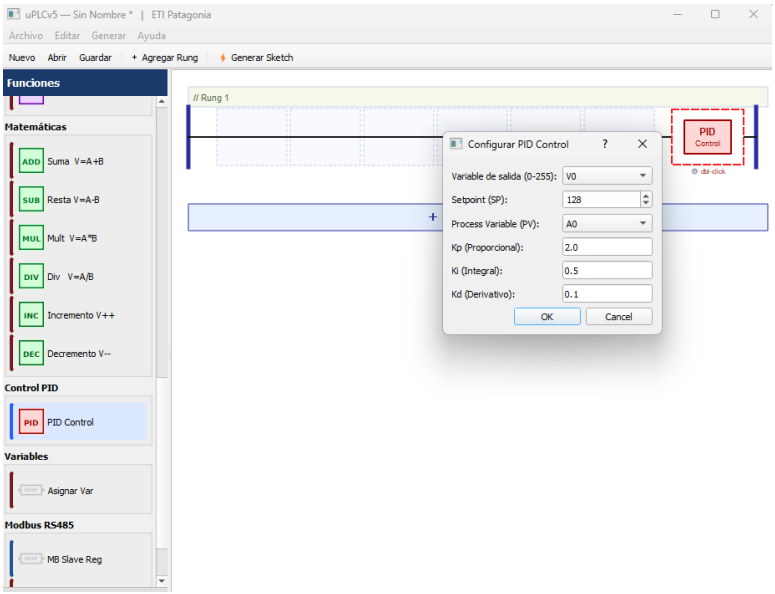
IMPORTANTE:
El reset es crítico para:
- Reiniciar máquina al encender
- Iniciar nuevo lote de producción
- Reset por condición (ej: al llegar a límite)

Ejemplo en ladder: En el siguiente ejemplo, veremos como trabajan los contadores y el bloque de reseteo de los contadores. La entrada I1 por cada pulso, incrementa C0. Cuando llega a la consigna configurada, enciende la salida Q1. La entrada I2, por cada pulso decrementa C1. Cuando llega al valor "cero", enciende la salida Q2. La necesidad de implementar el bloque de función RESET, radica en que cuando el contador llega a su consigna, queda congelado y no avanza ni retrocede mas; Es por ello que para que el programa pueda seguir trabajando con los contadores, debemos reiniciarlos. En el ejemplo, lo que hicimos fue sumar un bloque de función AND para que cuando ambos contadores terminen su ciclo, o cumplan su consigna, activen el registro de memoria M0 para que este mismo, luego interactúe con el bloque de función RESET.



🕮 Lección N°10 – PID

Configuración del Controlador PID:



El **PID** (*Proporcional-Integral-Derivativo*) es un control automático profesional para mantener una variable en un valor deseado.

Configuración:

- **Ref:** Variable de salida (V0-V9) → salida 0-255
- **SP (Setpoint):** Valor deseado (puede ser V o constante)
- **PV (Process Variable):** Variable actual (A0, A1, V)
- **Kp:** Ganancia Proporcional
- **Ki**: Ganancia Integral
- **Kd:** Ganancia Derivativa

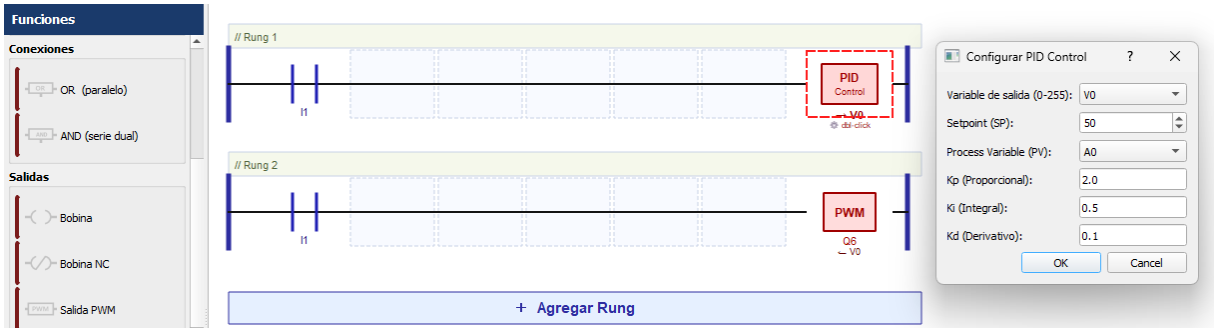
Ejemplo de sintaxis:

[PID:V0, SP=50,PV=A0,Kp=2.0,Ki=0.5,Kd=0.1]

Ejemplo 1: Control de Temperatura con PWM

Objetivo: Mantener temperatura en 50°C usando calefactor PWM

Hardware: Sensor temperatura → A0 + Calefactor → Q6 (PWM)
Programa:



Rung 1: [NO:I1=True] → [PID:V0, SP=50,PV=A0,Kp=2.0,Ki=0.5,Kd=0.1]
Rung 2: [NO:I1=True] → [PWM:Q6=V0]

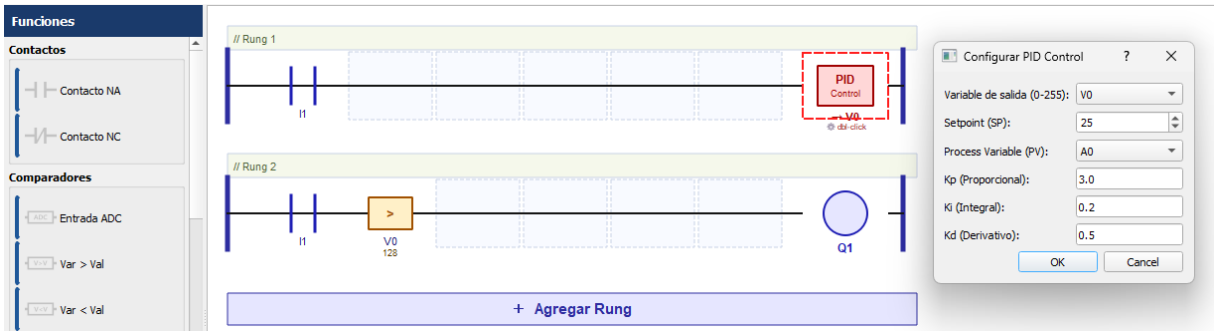
Explicación:

- I1 = enable del PID (usar ON siempre)
- PID calcula cuánta potencia necesita (0-255)
- Guarda resultado en V0
- PWM usa V0 para controlar calefactor

Si temperatura < 50°C → PID aumenta potencia
Si temperatura > 50°C → PID reduce potencia
Si temperatura = 50°C → PID mantiene estable

Ejemplo 2: Control ON/OFF con Histéresis

Objetivo: Controlar relé de forma inteligente (no solo ON/OFF brusco)
Programa:



Rung 1: [NO:I1=True] → [PID:V0, SP=25,PV=A0,Kp=3.0,Ki=0.2,Kd=0.5]
Rung 2: [NO:I1=True] → [V0 > 128] → [COIL:Q1]

Explicación:
• PID calcula necesidad de calefacción
• Si V0 > 128 (>50%), activa relé
• Histéresis natural del PID evita oscilaciones

Ajuste de Parámetros PID		
Kp (Proporcional): • Respuesta inmediata al error • Más alto = respuesta más rápida • Muy alto = oscilaciones	**Ki (Integral):** • Elimina error residual • Acumula error en el tiempo • Muy alto = inestabilidad	**Kd (Derivativo):** • Predice cambios futuros • Suaviza la respuesta • Muy alto = amplifica ruido
Valores recomendados iniciales: • Temperatura lenta: Kp=2.0, Ki=0.5, Kd=0.1 • Temperatura rápida: Kp=1.5, Ki=0.3, Kd=0.2 • Nivel líquido: Kp=1.0, Ki=0.1, Kd=0.05	**Método de ajuste:** 1. Empezar con Ki=0, Kd=0 2. Aumentar Kp hasta que oscile 3. Reducir Kp a la mitad 4. Agregar Ki lentamente 5. Agregar Kd si hay overshoot	

🎓 DESAFÍO FINAL: Diseñá e implementá un SISTEMA COMPLETO de tu elección.

- OPCIÓN A: INVERNADERO AUTOMÁTICO

Requisitos:

- Temperatura (A0): 15-30°C ideal
- Humedad tierra (A1): Mantener >50%
- Ventiladores (Q1): ON si T > 30°C
- Calefactor (Q2): ON si T < 15°C
- Riego (Q3): ON si humedad < 50%
- Luz artificial (Q4): ON si oscuro (I1 sensor luz)
- Alarma (Q5): Si temperatura crítica (>40°C o <5°C)
- Modo manual/auto (I2): Switch selector

- OPCIÓN B: CONTROL DE ACCESO

Requisitos:

- Teclado: 4 botones (I1-I4) para código PIN
- Código correcto: 1-2-3-4
- Motor puerta (Q1): Abre si código correcto
- Timer: Puerta cierra después de 10 seg
- Alarma (Q2): Si 3 intentos fallidos
- Reset (I5): Botón para resetear alarma
- LED verde (Q3): Acceso OK
- LED rojo (Q4): Acceso denegado

- OPCIÓN C: CINTA TRANSPORTADORA

Requisitos:

- START (I1): Botón arranque
- STOP (I2): Botón parada
- Emergencia (I3): Parada inmediata
- Sensor entrada (I4): Detecta pieza entrando
- Sensor salida (I5): Detecta pieza saliendo
- Motor cinta (Q1): Movimiento
- Contador (variable): Cuenta piezas procesadas
- Luz lleno (Q2): Aviso si 100 piezas
- Timer: Parada automática si inactiva 30 seg

🖊 TU PROYECTO… Elegí una opción (o inventá la tuya) y:

1. Dibujar diagrama en papel
2. Listar entradas y salidas
3. Escribir la lógica de cada rung
4. Implementar en uPLC
5. Testear paso a paso
6. Documentar con comentarios claros y con “tu nombre” lo subiremos a la carpeta de ejemplos de uPLC

✅ CRITERIOS DE EVALUACIÓN… Tu proyecto es exitoso si:

✅ Funciona según especificaciones

✅ Tiene comentarios claros

✅ Maneja casos de error

✅ Tiene protecciones de seguridad

✅ Es fácil de entender para otro programador

¡¡¡Si llegaste a este punto, FELICITACIONES!!!
¡Completaste el curso!

Ahora sabés:

- ✅ Lógica básica (AND, OR,NOT,EXOR)
- ✅ Temporizadores
- ✅ Entradas analógicas
- ✅ Memorias y retención
- ✅ Comunicación Modbus
- ✅ Proyectos completos

PRÓXIMOS PASOS:

1. Practicar con los ejemplos en /ejemplos
2. Experimentar con nuestro hardware (uPLC)
3. Compartir tus proyectos

RECURSOS:

📖 MANUAL_LADDER_COMPLETO.txt

📖 MODBUS_GUIDE.md

📖 EJEMPLO_MODBUS.md

📖 HARDWARE_RS485.md

CONTACTO

ETI Patagonia

prof.martintorres@educ.ar

https://eti-patagonia-ar.github.io/ETI-Patagonia-ARG/

¡Seguí aprendiendo y automatizando!

- FIN DEL CURSO –

ARDUINO
ETI PATAGONIA
prof.martintorres@educ.ar